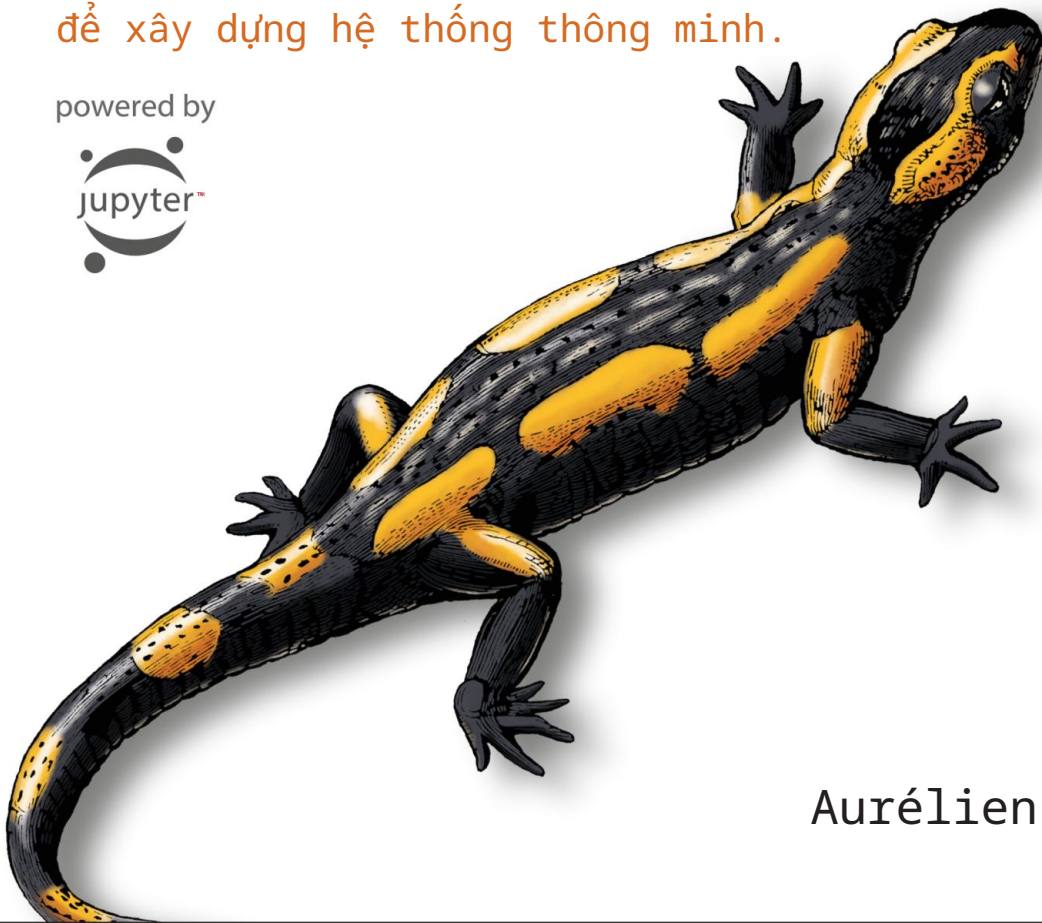


O'REILLY®

Thực hành Học máy với Scikit- Learn, Keras & TensorFlow

Các khái niệm, công cụ và kỹ thuật
để xây dựng hệ thống thông minh.

powered by



Aurélien Géron

O'REILLY®

Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow

Through a series of breakthroughs, Deep Learning has boosted the entire field of machine learning. Now, even programmers who know close to nothing about this technology can use simple, efficient tools to implement programs capable of learning from data. The updated edition of this best-selling book uses concrete examples, minimal theory, and production-ready Python frameworks to help you gain an intuitive understanding of the concepts and tools for building intelligent systems.

You'll learn a range of techniques that you can quickly put to use. With exercises in each chapter to help you apply what you've learned, all you need is programming experience to get started. All code is available on GitHub. It has been updated to TensorFlow 2 and the latest version of Scikit-Learn.

- Learn Machine Learning fundamentals through an end-to-end project using Scikit-Learn and pandas
- Build and train many neural network architectures for classification and regression using TensorFlow 2
- Discover object detection, semantic segmentation, attention mechanisms, language models, GANs, and more
- Explore the Keras API, the official high-level API for TensorFlow 2
- Productionize TensorFlow models using TensorFlow's Data API, distribution strategies API, TF Transform, and TF-Serving
- Deploy on Google Cloud AI Platform or on mobile devices
- Exploit unsupervised learning techniques such as dimensionality reduction, clustering, and anomaly detection
- Create autonomous learning agents with Reinforcement Learning, including using the TF-Agents library

"An exceptional resource to study Machine Learning. You will find clear-minded, intuitive explanations, and a wealth of practical tips."

—François Chollet

Author of Keras, author of *Deep Learning with Python*

"This book is a great introduction to the theory and practice of solving problems with neural networks; I recommend it to anyone interested in learning about practical ML."

—Pete Warden

Mobile Lead for TensorFlow

Aurélien Géron is a machine learning consultant and trainer. A former Googler, he led YouTube's video classification team from 2013 to 2016. He was also a founder and CTO of Wifirst (a leading Wireless ISP in France) from 2002 to 2012.

DATA | DATA SCIENCE | DATA ANALYTICS
MACHINE LEARNING | DEEP LEARNING
PYTHON MACHINE LEARNING

US \$74.99

CAN \$98.99

ISBN: 978-1-492-03264-9



Twitter: @oreillymedia
facebook.com/oreilly

ẤN BẢN THỨ HAI

Thực hành máy học với Scikit-Learn, Keras và TensorFlow

Các khái niệm, công cụ và kỹ thuật để
Xây dựng hệ thống thông minh

Aurélien Géron

Bắc Kinh Boston Farnham • Sebastopol • Tokyo

O'REILLY®

Thực hành máy học với Scikit-Learn, Keras và TensorFlow
bởi Aurélien Géron

Bản quyền © 2019 Kiwisoft SAS. Mọi quyền được bảo lưu.

In tại Canada.

Xuất bản bởi O'Reilly Media, Inc., 1005 Gravenstein Highway North, Sebastopol, CA 95472.

Sách của O'Reilly có thể được mua để sử dụng cho mục đích giáo dục, kinh doanh hoặc quảng bá bán hàng. Phiên bản trực tuyến cũng có sẵn cho hầu hết các đầu sách (<http://oreilly.com>). Để biết thêm thông tin chi tiết, vui lòng liên hệ bộ phận bán hàng doanh nghiệp/tổ chức của chúng tôi: 800-998-9938 hoặc corporate@oreilly.com.

Biên tập viên: Rachel Roumeliotis và Nicole Tache

Người lập chỉ mục: Judith McConville

Biên tập viên sản xuất: Kristen Brown

Nhà thiết kế nội thất: David Futato

Biên tập viên: Amanda Kersey

Thiết kế bìa: Karen Montgomery

Người soát lỗi: Rachel Head

Họa sĩ minh họa: Rebecca Demarest

Tháng 9 năm 2019: Ấn bản thứ hai

Lịch sử sửa đổi cho ấn bản thứ hai

05/09/2019: Phát hành lần đầu

11/10/2019: Bản phát hành thứ hai

22/11/2019: Bản phát hành thứ ba

Xem <http://oreilly.com/catalog/errata.csp?isbn=9781492032649> để biết thêm chi tiết về ngày phát hành.

Logo của O'Reilly là nhãn hiệu đã được đăng ký của O'Reilly Media, Inc. "Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow", hình ảnh bìa và các hình thức thương mại liên quan đều là nhãn hiệu của O'Reilly Media, Inc.

Các quan điểm được thể hiện trong tác phẩm này là của tác giả và không phản ánh quan điểm của nhà xuất bản. Mặc dù nhà xuất bản và tác giả đã nỗ lực hết sức để đảm bảo thông tin và hướng dẫn trong tác phẩm này là chính xác, nhà xuất bản và tác giả từ chối mọi trách nhiệm đối với các lỗi hoặc thiếu sót, bao gồm nhưng không giới hạn trách nhiệm đối với thiệt hại phát sinh từ việc sử dụng hoặc dựa vào tác phẩm này. Việc sử dụng thông tin và hướng dẫn trong tác phẩm này là rủi ro của riêng bạn. Nếu bất kỳ đoạn mã hoặc công nghệ nào khác mà tác phẩm này chứa đựng hoặc mô tả thuộc giấy phép mã nguồn mở hoặc quyền sở hữu trí tuệ của người khác, bạn có trách nhiệm đảm bảo việc sử dụng chúng tuân thủ các giấy phép và/hoặc quyền đó.

Mục lục

Lời tựa. ...	xv
Phần I. Những nguyên lý cơ bản của máy học	
1. Bức tranh tổng quan về Học máy. ...	
Máy học là gì?	2
Tại sao nên sử dụng máy học?	2
Ví dụ về các ứng dụng	5
Các loại hệ thống học máy Học có	7
giám sát/không giám sát Học theo	7
lô và học trực tuyến Học	14
dựa trên trường hợp so với học dựa trên mô hình	17
Những thách thức chính của học máy	23
Số lượng dữ liệu huấn luyện không đủ	23
Dữ liệu huấn luyện không mang tính đại diện	25
Dữ liệu chất lượng kém	26
Các tính năng không liên quan	27
Hiện tượng quá khớp dữ liệu huấn luyện	27
Hiện tượng thiếu khớp dữ liệu huấn luyện	29
Lỗi lại	30
Kiểm thử và xác nhận	30
Điều chỉnh siêu tham số và lựa chọn mô hình	31
Dữ liệu không khớp	32
Bài tập	33
2. Dự án Học máy từ đầu đến cuối. ...	
Làm việc với dữ liệu thực tế	35

Hãy nhìn vào bức tranh toàn cảnh	37
Xác định vấn đề	37
Chọn một thước đo hiệu suất	39
Kiểm tra các giả định	42
Lấy dữ liệu	42
Tạo không gian làm việc	42
Tải xuống dữ liệu	46
Hãy cùng xem qua cấu trúc dữ liệu.	47
Tạo bộ kiểm thử	51
Khám phá và trực quan hóa dữ liệu để thu được những hiểu biết sâu sắc.	56
Trực quan hóa dữ liệu địa lý	56
Tìm kiếm mối tương quan	58
Thử nghiệm các tổ hợp thuộc tính	61
Chuẩn bị dữ liệu cho các thuật toán học máy	62
Làm sạch dữ liệu	63
Xử lý các thuộc tính văn bản và phân loại	65
Máy biến áp tùy chỉnh	68
Mở rộng tính năng	69
Các đường ống chuyển đổi	70
Chọn và huấn luyện một mô hình	72
Huấn luyện và đánh giá trên tập dữ liệu huấn luyện	72
Đánh giá tốt hơn bằng phương pháp kiểm định chéo	73
Tính chính mô hình của bạn	75
Tìm kiếm lưới	76
Tìm kiếm ngẫu nhiên	78
Phương pháp tập hợp	78
Phân tích các mô hình tốt nhất và lỗi của chúng	78
Đánh giá hệ thống của bạn trên tập dữ liệu	79
kiểm thử Khởi chạy, giám sát và bảo trì hệ thống của	80
bạn Hãy thử ngay!	83
Bài tập	84

3. Phân loại. ...

MNIST	85
Huấn luyện bộ phân loại nhị phân	88
Các biện pháp đánh giá hiệu suất	88
Đo lường độ chính xác bằng phương pháp kiểm định chéo	89
Ma trận nhầm lẫn	90
Độ chính xác và khả năng thu hồi	92
Sự đánh đổi giữa độ chính xác và độ thu hồi	93
Đường cong ROC	97
Phân loại đa lớp	100

Phân tích lỗi	102
Phân loại đa nhãn	106
Phân loại đa đầu ra	107
Bài tập	108
 4. Mô hình huấn luyện. ...	 111
Hồi quy tuyến tính	112
Phương trình chuẩn Độ phức	114
tập tính toán Thuật toán giảm độ	117
dốc	118
Giảm độ dốc theo lô	121
Thuật toán giảm độ dốc ngẫu nhiên (Stochastic Gradient Descent)	124
Giảm độ dốc theo lô nhỏ	127
Hồi quy đa thức	128
Đường cong học tập	130
Mô hình tuyến tính được điều chỉnh	134
Hồi quy Ridge	135
Lasso Regression	137
Lưới đàn hồi	140
Dừng sớm	141
Hồi quy Logistic	142
Ước tính xác suất	143
Chức năng đào tạo và chi phí	144
Ranh giới quyết định	145
Hồi quy Softmax	148
Bài tập	151
 5. Máy hỗ trợ vectơ (Support Vector Machines)	 153
Phân loại SVM tuyến tính	153
Phân loại lề mềm	154
Phân loại SVM phi tuyến tính	157
Hạt nhân đa thức	158
Đặc điểm tương đồng	159
Hạt nhân RBF Gaussian	160
Độ phức tạp tính toán	162
Hồi quy SVM	162
Bên trong nắp ca-pô	164
Hàm quyết định và dự đoán	165
Mục tiêu đào tạo	166
Lập trình bậc hai	167
Vấn đề kép	168
SVM nhân hóa	169

SVM trực tuyến	172
Bài tập	174
6. Cây quyết định. ...	
Huấn luyện và trực quan hóa cây quyết định	175
Đưa ra dự đoán Ước	176
tính xác suất lớp Thuật toán	178
huấn luyện CART Độ phức tạp tính	179
toán Gini Độ không thuần	180
khuyết hay entropy?	180
Siêu tham số điều chỉnh Sự bất ổn	181
của hồi quy	183
	185
Bài tập	186
7. Học tập kết hợp và Rừng ngẫu nhiên. 189	
Phân loại bỏ phiếu, Phương	189
pháp đóng gói và dán (Bagging and	192
Pasteing), Phương pháp đóng gói và dán trong Scikit-Learn,	194
Đánh giá ngoài tập dữ liệu (Out-of-	195
Bag Evaluation), Các mảng ngẫu nhiên và không gian con ngẫu nhiên,	196
Rừng ngẫu nhiên (Random Forests).	197
Extra-Trees	198
Tầm quan trọng của tính năng	198
Tăng cường	199
AdaBoost	200
Tăng cường độ dốc	203
Xếp chồng	208
Bài tập	211
8. Giảm chiều dữ liệu. ...	
Lời nguyên của chiều không gian	214
Các phương pháp chính để giảm chiều dữ liệu	215
Phép chiếu	215
Học tập đa dạng	218
PCA	219
Bảo toàn phương sai	219
Các thành phần chính	220
Chiều xuống các chiều d	221
Sử dụng Scikit-Learn	222
Tỷ lệ phương sai được giải thích	222
Lựa chọn số chiều phù hợp	223

PCA để nén	224
PCA ngẫu nhiên	225
PCA gia tăng	225
PCA hạt nhân	226
Lựa chọn nhân hệ điều hành và tinh chỉnh các siêu tham số	227
LLE	230
Các kỹ thuật giảm chiều khác	232
Bài tập	233

9. Các kỹ thuật học không giám sát. ...

Phân cụm	236
K-Means	238
Giới hạn của thuật toán K-Means	248
Sử dụng thuật toán phân cụm để phân đoạn ảnh	249
Sử dụng phân cụm để tiền xử lý dữ liệu	251
Sử dụng phân cụm cho học bán giám sát	253
DBSCAN	255
Các thuật toán phân cụm khác	258
Hỗn hợp Gaussian	260
Phát hiện bất thường bằng cách sử dụng hỗn hợp Gaussian	266
Chọn số lượng cụm	267
Mô hình hỗn hợp Gaussian Bayes	270
Các thuật toán khác để phát hiện bất thường và mới lạ	274
Bài tập	275

Phần II. Mạng nơ-ron và học sâu

10. Giới thiệu về Mạng nơ-ron nhân tạo với Keras.	279
Từ tế bào thần kinh sinh học đến tế bào thần kinh nhân tạo	280
Các tế bào thần kinh sinh học	281
Tính toán logic với nơ-ron	283
Perceptron	284
Perceptron đa lớp và lan truyền ngược	289
Mạng nơ-ron truyền đa lớp hồi quy (Regression MLP)	292
Phân loại MLP	294
Triển khai MLP bằng Keras	295
Cài đặt TensorFlow 2	296
Xây dựng bộ phân loại hình ảnh sử dụng API tuần tự	297
Xây dựng mạng nơ-ron đa lớp hồi quy (MLP) bằng API tuần tự.	307
Xây dựng các mô hình phức tạp bằng cách sử dụng API chức năng	308
Sử dụng API kế thừa lớp con để xây dựng các mô hình động	313

Lưu trữ và khôi phục mô hình	314
Sử dụng hàm gọi lại	315
Sử dụng TensorBoard để trực quan hóa	317
Tính chỉnh các siêu tham số của mạng nơ-ron	320
Số lượng lớp ẩn	323
Số lượng nơron trên mỗi lớp ẩn	324
Tốc độ học, kích thước lô và các siêu tham số khác	325
Bài tập	327
 11. Huấn luyện mạng nơ-ron sâu. ...	
Các vấn đề về độ dốc biến mất/bùng nổ	332
Khởi tạo Glorot và He	333
Hàm kích hoạt không bão hòa	335
Chuẩn hóa theo lô	338
Cắt xén gradient	345
Tái sử dụng các lớp đã được huấn luyện trước	345
Chuyển giao học tập với Keras	347
Huấn luyện trước không giám sát	349
Huấn luyện trước về một nhiệm vụ phụ trợ	350
Bộ tối ưu hóa nhanh hơn	351
Tối ưu hóa động lượng	351
Phương pháp gia tốc gradient Nesterov	353
AdaGrad	354
RMSProp	355
Tối ưu hóa Adam và Nadam	356
Lập lịch tốc độ học tập	359
Tránh hiện tượng quá khớp dữ liệu thông qua phương pháp điều chỉnh (Regularization).	364
Chuẩn hóa ℓ_1 và ℓ_2	364
Bỏ học	365
Bỏ học Monte Carlo (MC)	368
Chuẩn hóa Max-Norm	370
Tóm tắt và hướng dẫn thực hành	371
Bài tập	373
 12. Mô hình tùy chỉnh và huấn luyện với TensorFlow.	375
Tổng quan nhanh về TensorFlow	376
Sử dụng TensorFlow tương tự như NumPy	379
Tensor và các phép toán	379
Tensor và NumPy	381
Chuyển đổi kiểu dữ liệu	381
Biến số	382
Các cấu trúc dữ liệu khác	383

Tùy chỉnh mô hình và thuật toán huấn luyện	384
Hàm mất mát tùy chỉnh	384
Lưu và tải các mô hình chứa các thành phần tùy chỉnh	385
Các hàm kích hoạt tùy chỉnh, hàm khởi tạo, hàm điều chỉnh và các ràng buộc	387
Số liệu tùy chỉnh	388
Lớp tùy chỉnh	391
Mô hình tùy chỉnh	394
Tồn thất và các chỉ số dựa trên cấu trúc bên trong mô hình	397
Tính toán độ dốc bằng phương pháp tự động vi phân	399
Các vòng lặp huấn luyện tùy chỉnh	402
Các hàm và đồ thị trong TensorFlow	405
AutoGraph và Tracing	407
Quy tắc chức năng TF	409
Bài tập	410
13. Tải và tiền xử lý dữ liệu với TensorFlow.	413
API dữ liệu	414
Chuỗi các phép biến đổi Xáo trộn	415
dữ liệu Tiền xử lý dữ	416
liệu Kết hợp mọi thứ Tải	419
trước dữ liệu Sử dụng tập dữ liệu	420
với tf.keras	421
Định dạng TFRecord Tập TFRecord nén	423
Giới thiệu ngắn gọn về	424
Protocol Buffers TensorFlow	425
Protobufs	425
Ví dụ về tải và phân tích cú pháp	427
Xử lý danh sách các danh sách bằng cách sử dụng Protobuf SequenceExample	428
Xử lý sơ bộ các đặc trưng đầu vào	429
Mã hóa các đặc trưng phân loại bằng cách sử dụng vectơ One-Hot	430
Mã hóa các đặc trưng phân loại bằng cách sử dụng phép nhúng	431
Các lớp tiền xử lý Keras	433
Biến đổi TF	437
Dự án Bộ dữ liệu TensorFlow (TFDS)	439
Bài tập	441
14. Thị giác máy tính sâu sử dụng mạng nơ-ron tích chập.	442
Kiến trúc của vỏ não thị giác: Các lớp tích	446
chập, bộ lọc, xếp chồng	448
nhiều	450
bản đồ đặc trưng.	451

Triển khai TensorFlow	453
Yêu cầu bộ nhớ	456
Lớp gộp	456
Triển khai TensorFlow	458
Kiến trúc CNN	460
LeNet-5	463
AlexNet	464
GoogleLeNet	466
VGGNet	470
ResNet	471
Xception	474
SENet	476
Triển khai mạng CNN ResNet-34 bằng Keras	478
Sử dụng các mô hình được huấn luyện trước từ Keras	479
Mô hình được huấn luyện trước cho học chuyển giao	481
Phân loại và định vị	483
Phát hiện đối tượng	485
Mạng tích chập hoàn toàn	487
Bạn chỉ nhìn một lần thôi (YOLO)	489
Phân đoạn ngữ nghĩa	492
Bài tập	496
15. Xử lý chuỗi bằng RNN và CNN.	497
Nơ-ron hồi quy và các lớp, ô nhớ, Chuỗi	498
đầu vào và đầu ra,	500
Huấn luyện RNN, Dự báo chuỗi thời	501
gian, Các chỉ số cơ	502
bản, Triển khai RNN đơn giản, RNN	503
sâu, Dự báo nhiều	505
bước thời gian phía trước, Xử lý chuỗi	505
dài, Khắc phục	506
vấn đề độ dốc không ổn định, Giải quyết vấn đề bộ	508
nhớ ngắn hạn, Bài tập.	511
	512
	514
	523
16. Xử lý ngôn ngữ tự nhiên với mạng nơ-ron hồi quy (RNN) và cơ chế chú ý.	525
Tạo văn bản Shakespeare bằng cách sử dụng mạng nơ-ron nhân vật (Character RNN).	526
Tạo tập dữ liệu huấn luyện	527
Cách chia tách một tập dữ liệu tuần tự	527
Chia tập dữ liệu tuần tự thành nhiều cửa sổ	528

Xây dựng và huấn luyện mô hình Char-RNN	530
Sử dụng mô hình Char-RNN	531
Tạo văn bản giả mạo của Shakespeare	531
RNN có trạng thái	532
Phân tích cảm xúc	534
Che giấu	538
Tái sử dụng các embedding đã được huấn luyện trước	540
Mạng mã hóa-giải mã cho dịch máy thần kinh	542
RNN hai chiều	546
Tìm kiếm chùm tia	547
Cơ chế chú ý	549
Sự chú ý thị giác	552
Chỉ cần sự chú ý là đủ: Kiến trúc Transformer	554
Những đổi mới gần đây trong mô hình ngôn ngữ	563
Bài tập	565
 17. Học biểu diễn và học tạo sinh sử dụng bộ mã hóa tự động và GAN.	567
Biểu diễn dữ liệu hiệu quả	569
Thực hiện phân tích thành phần chính (PCA) với bộ mã hóa tự động tuyến tính chưa hoàn chỉnh.	570
Bộ mã hóa tự động xếp chồng	572
Triển khai Autoencoder xếp chồng bằng Keras	572
Hình dung quá trình tái tạo	574
Trực quan hóa bộ dữ liệu MNIST về thời trang	574
Huấn luyện trước không giám sát bằng cách sử dụng bộ mã hóa tự động xếp chồng (Stacked Autoencoders)	576
Trọng lượng buộc	577
Huấn luyện từng bộ mã hóa tự động một.	578
Bộ mã hóa tự động tích chập	579
Bộ mã hóa tự động hồi quy	580
Bộ mã hóa tự động khử nhiễu	581
Bộ mã hóa tự động thừa thớt	582
Bộ mã hóa tự động biến phân	586
Tạo ảnh MNIST về thời trang	590
Mạng đối kháng tạo sinh	592
Những khó khăn trong việc huấn luyện GAN	596
GAN tích chập sâu	598
Sự phát triển dần dần của GAN	601
StyleGANs	604
Bài tập	607
 18. Học tăng cường. ...	
Học cách tối ưu hóa tìm kiếm chính	610
sách phần thưởng	612

Giới thiệu về OpenAI Gym	613
Chính sách mạng thần kinh	617
Đánh giá hành động: Vấn đề phân bổ tín dụng	619
Độ dốc chính sách	620
Quá trình quyết định Markov	625
Học tập chênh lệch thời gian	629
Q-Learning	630
Chính sách thăm dò	632
Q-Learning xấp xỉ và Q-Learning sâu	633
Triển khai Deep Q-Learning	634
Các biến thể của Deep Q-Learning	639
Mục tiêu giá trị Q cố định	639
Double DQN	640
Phát lại trải nghiệm ưu tiên	640
Đầu DQN	641
Thư viện TF-Agents	642
Cài đặt TF-Agents	643
Môi trường TF-Agents	643
Thông số kỹ thuật môi trường	644
Các trình bao bọc môi trường và quá trình tiền xử lý của Atari	645
Kiến trúc đào tạo	649
Tạo ra mạng Q sâu	650
Tạo tác nhân DQN	652
Tạo bộ đệm phát lại và bộ quan sát tương ứng.	654
Xây dựng các chỉ số đo lường hiệu quả đào tạo	655
Tạo trình điều khiển Collect	656
Tạo tập dữ liệu	658
Tạo vòng lặp huấn luyện	661
Tổng quan về một số thuật toán học tăng cường phổ biến	662
Bài tập	664
19. Huấn luyện và triển khai các mô hình TensorFlow ở quy mô lớn.	667
Phục vụ một mô hình TensorFlow	668
Sử dụng TensorFlow Serving	668
Xây dựng dịch vụ dự đoán trên nền tảng GCP AI Platform.	677
Sử dụng Dịch vụ Dự đoán	682
Triển khai mô hình lên thiết bị di động hoặc thiết bị nhúng	685
Sử dụng GPU để tăng tốc độ tính toán	689
Sở hữu GPU của riêng bạn	690
Sử dụng máy ảo được trang bị GPU	692
Phòng thí nghiệm hợp tác	693
Quản lý RAM GPU	694

Đặt các phép toán và biến trên thiết bị	697
Thực thi song song trên nhiều thiết bị	699
Huấn luyện mô hình trên nhiều thiết bị Song song hóa	701
mô hình Song song hóa	701
dữ liệu Huấn luyện	704
quy mô lớn bằng API Chiến lược phân phối Huấn luyện mô hình trên cụm	709
TensorFlow Chạy các tác vụ huấn luyện lớn trên Google	711
Cloud AI Platform Điều chỉnh siêu tham số hộp đen trên AI Platform Bài tập	714
Cảm ơn!	716
	717
	718
A. Đáp án bài tập. ...	
B. Danh sách kiểm tra dự án học máy. ...	
C. Bài toán kép SVM. ...	
D. Tự động phân biệt. 765	
E. Các kiến trúc ANN phổ biến khác. ...	
F. Cấu trúc dữ liệu đặc biệt. ...	
G. Đồ thị TensorFlow. ...	
Mục lục. ...	

Lời tựa

Cơn sóng thần học máy

Năm 2006, Geoffrey Hinton và cộng sự đã công bố **một bài báo**¹ Bài báo này trình bày cách huấn luyện một mạng nơ-ron sâu có khả năng nhận dạng chữ số viết tay với độ chính xác hàng đầu (>98%). Họ đặt tên cho kỹ thuật này là “Học sâu” (Deep Learning). Mạng nơ-ron sâu là một mô hình (rất) đơn giản hóa của vỏ não chúng ta, bao gồm một chồng các lớp nơ-ron nhân tạo. Việc huấn luyện một mạng nơ-ron sâu được cho là bất khả thi vào thời điểm đó, và hầu hết các nhà nghiên cứu đã từ bỏ ý tưởng này vào cuối những năm 1990. Bài báo này đã khơi dậy sự quan tâm của cộng đồng khoa học, và chẳng bao lâu sau, nhiều bài báo mới đã chứng minh rằng Học sâu không chỉ khả thi mà còn có khả năng đạt được những thành tựu đáng kinh ngạc mà không một kỹ thuật Học máy (ML) nào khác có thể sánh kịp (với sự trợ giúp của sức mạnh tính toán khổng lồ và lượng dữ liệu lớn). Sự nhiệt tình này nhanh chóng lan rộng sang nhiều lĩnh vực khác của Học máy.

Khoảng một thập kỷ sau, Học máy đã chinh phục ngành công nghiệp: nó là cốt lõi của phần lớn sự kỳ diệu trong các sản phẩm công nghệ cao ngày nay, xếp hạng kết quả tìm kiếm trên web của bạn, hỗ trợ nhận dạng giọng nói trên điện thoại thông minh, đề xuất video và thậm chí đánh bại nhà vô địch thế giới trong trò chơi cờ vây. Chẳng mấy chốc, nó sẽ lái xe của bạn.

Ứng dụng Học máy trong Dự án của Bạn

Vậy nên, chắc hẳn bạn rất hào hứng với Máy học và muốn tham gia vào lĩnh vực này!

1 Geoffrey E. Hinton et al., “Một thuật toán học nhanh cho mạng niềm tin sâu,” *Tính toán thần kinh* 18 (2006): 1527-1554.

2. Mặc dù mạng nơ-ron tích chập sâu của Yann LeCun đã hoạt động tốt trong việc nhận dạng hình ảnh. Kể từ những năm 1990, mặc dù chúng không phải là loại đa năng như vậy.

Có lẽ bạn muốn trang bị cho con robot tự chế của mình một bộ não riêng? Giúp nó nhận diện khuôn mặt? Hay học cách đi lại?

Hoặc có thể công ty của bạn có rất nhiều dữ liệu (nhật ký người dùng, dữ liệu tài chính, dữ liệu sản xuất, dữ liệu cảm biến máy móc, số liệu thống kê đường dây nóng, báo cáo nhân sự, v.v.), và rất có thể bạn có thể tìm thấy một số thông tin quý giá nếu biết cách tìm kiếm. Với Học máy, bạn có thể thực hiện được những điều sau đây **và hơn thế nữa**:

- Phân khúc khách hàng và tìm ra chiến lược tiếp thị tốt nhất cho từng nhóm.
- Đề xuất sản phẩm cho từng khách hàng dựa trên những gì khách hàng tương tự đã mua.
- Phát hiện những giao dịch có khả năng gian lận.
- Dự báo doanh thu năm tới.

Dù lý do là gì, bạn đã quyết định học Máy học và áp dụng nó vào các dự án của mình. Thật là một ý tưởng tuyệt vời!

Mục tiêu và phương pháp tiếp cận

Cuốn sách này giả định rằng bạn hầu như không biết gì về Học máy. Mục tiêu của nó là cung cấp cho bạn các khái niệm, công cụ và trực giác cần thiết để triển khai các chương trình có khả năng học hỏi từ dữ liệu.

Chúng ta sẽ tìm hiểu một lượng lớn các kỹ thuật, từ những kỹ thuật đơn giản và phổ biến nhất (như Hồi quy tuyến tính) đến một số kỹ thuật Học sâu thường xuyên giành chiến thắng trong các cuộc thi.

Thay vì tự xây dựng các phiên bản thử nghiệm của từng thuật toán, chúng ta sẽ sử dụng các framework Python sẵn có cho môi trường sản xuất:

- **Scikit-Learn** Nó rất dễ sử dụng, nhưng lại triển khai nhiều thuật toán Học máy một cách hiệu quả, vì vậy nó là một điểm khởi đầu tuyệt vời để học Học máy. Nó được David Cournapeau tạo ra vào năm 2007, và hiện đang được dẫn dắt bởi một nhóm các nhà nghiên cứu tại Viện Nghiên cứu Khoa học Máy tính và Tự động hóa Pháp (Inria).
- **TensorFlow** TensorFlow (TF) là một thư viện phức tạp hơn dành cho tính toán số phân tán. Nó cho phép huấn luyện và chạy các mạng nơ-ron rất lớn một cách hiệu quả bằng cách phân tán các phép tính trên hàng trăm máy chủ đa GPU (bộ xử lý đồ họa). TF được tạo ra tại Google và hỗ trợ nhiều ứng dụng Học máy quy mô lớn của công ty này. Nó được công khai mã nguồn vào tháng 11 năm 2015, và phiên bản 2.0 được phát hành vào tháng 9 năm 2019.
- **Keras** là một API học sâu cấp cao giúp việc huấn luyện và chạy mạng nơ-ron trở nên rất đơn giản. Nó có thể chạy trên nền tảng TensorFlow, Theano hoặc Microsoft Cognitive Toolkit (trước đây gọi là CNTK). TensorFlow đi kèm với API của nó.

Đây là một triển khai riêng của API này, được gọi là `tf.keras`, cung cấp hỗ trợ cho một số tính năng nâng cao của TensorFlow (ví dụ: khả năng tải dữ liệu hiệu quả).

Cuốn sách ưu tiên phương pháp thực hành, giúp người đọc hiểu sâu sắc về Học máy thông qua các ví dụ cụ thể và một chút lý thuyết.

Mặc dù bạn có thể đọc cuốn sách này mà không cần dùng đến máy tính xách tay, tôi vẫn đặc biệt khuyên bạn nên thử nghiệm với các ví dụ mã có sẵn trực tuyến dưới dạng sổ tay Jupyter tại <https://github.com/ageron/handson-ml2>.

Điều kiện tiên quyết

Cuốn sách này giả định rằng bạn đã có một số kinh nghiệm lập trình Python và quen thuộc với các thư viện khoa học chính của Python—đặc biệt là `NumPy`, `gấu trúc`, và `Matplotlib`.

Ngoài ra, nếu bạn quan tâm đến những gì diễn ra bên trong máy tính, bạn cũng nên có hiểu biết tương đối tốt về toán học trình độ đại học (giải tích, đại số tuyến tính, xác suất và thống kê).

Nếu bạn chưa biết Python, hãy truy cập <http://learnpython.org/> Đây là một nơi tuyệt vời để bắt đầu.

Hướng dẫn chính thức trên `Python.org` Cũng khá tốt.

Nếu bạn chưa từng sử dụng Jupyter, [Chương 2](#) sẽ hướng dẫn bạn cài đặt và những kiến thức cơ bản: đây là một công cụ mạnh mẽ nên có trong bộ công cụ của bạn.

Nếu bạn chưa quen với các thư viện khoa học của Python, các tài liệu hướng dẫn Jupyter được cung cấp bao gồm một vài bài hướng dẫn. Ngoài ra còn có một bài hướng dẫn toán học ngắn gọn về đại số tuyến tính.

Lộ trình

Cuốn sách này được chia thành hai phần. [Phần I, Những nguyên lý cơ bản của học máy](#), bao gồm các chủ đề sau:

- Máy học là gì, nó cố gắng giải quyết những vấn đề gì và các loại chính...

Các khái niệm cơ bản và lý thuyết của hệ thống • Các bước

trong một dự án Học máy điển hình • Học bằng cách khớp mô

hình với dữ liệu • Tối ưu hóa hàm chi phí •

Xử lý, làm sạch và chuẩn bị dữ liệu

- Lựa chọn và thiết kế các đặc trưng • Lựa chọn

mô hình và tinh chỉnh siêu tham số bằng

phương pháp kiểm định chéo • Những thách thức của Học máy, đặc biệt là hiện tượng thiếu

khớp và quá khớp

(Sự đánh đổi giữa độ lệch và phương sai)

- Các thuật toán học máy phổ biến nhất: Hồi quy tuyến tính và hồi quy đa thức, Hồi quy Logistic, Thuật toán k-Hàng xóm gần nhất, Máy vectơ hỗ trợ, Quyết định Cây quyết định, Rừng ngẫu nhiên và Phương pháp tập hợp
- Giảm số chiều của dữ liệu huấn luyện để chống lại "lời nguyền về số chiều" tính cá nhân"
- Các kỹ thuật học không giám sát khác, bao gồm phân cụm, ước lượng mật độ,... phát hiện dị thường và bất thường

Phần II, Mạng thần kinh và Học sâu, bao gồm các chủ đề sau:

- Mạng nơ-ron là gì và chúng có ích trong những lĩnh vực

nào • Xây dựng và huấn luyện mạng nơ-ron bằng TensorFlow và Keras • Các

kiến trúc mạng nơ-ron quan trọng nhất: mạng nơ-ron truyền thẳng cho dữ liệu dạng bảng, mạng nơ-ron tích chập cho thị giác máy tính, mạng nơ-ron hồi quy và mạng bộ nhớ dài hạn ngắn hạn (LSTM) cho xử lý chuỗi, bộ mã hóa/giải mã và bộ biến đổi cho xử lý ngôn ngữ tự nhiên, bộ mã hóa tự động và mạng đối kháng tạo sinh (GAN) cho học tạo sinh • Các kỹ thuật huấn luyện mạng nơ-ron sâu • Cách xây dựng một tác nhân (ví dụ: một bot trong

trò chơi) có thể học các chiến lược tốt thông qua

thử và sai, sử dụng Học tăng cường • Tải và tiền xử lý lượng lớn dữ liệu một cách hiệu quả • Huấn luyện và triển khai các mô hình TensorFlow ở quy mô lớn

Phần đầu chủ yếu dựa trên Scikit-Learn, trong khi phần thứ hai sử dụng TensorFlow và Keras.



Đừng vội vàng lao vào vùng nước sâu: Mặc dù Học sâu (Deep Learning) chắc chắn là một trong những lĩnh vực thú vị nhất trong Học máy (Machine Learning), bạn nên nắm vững các nguyên tắc cơ bản trước. Hơn nữa, hầu hết các vấn đề có thể được giải quyết khá tốt bằng các kỹ thuật đơn giản hơn như Rừng ngẫu nhiên (Random Forests) và các phương pháp kết hợp (Ensemble methods) (đã thảo luận trong **Phần I**). Học sâu phù hợp nhất cho các bài toán phức tạp như nhận dạng hình ảnh, nhận dạng giọng nói hoặc xử lý ngôn ngữ tự nhiên, miễn là bạn có đủ dữ liệu, sức mạnh tính toán và sự kiên nhẫn.

Những thay đổi trong ấn bản thứ hai

Phiên bản thứ hai này có sáu mục tiêu chính:

1. Nội dung bao gồm các chủ đề ML bổ sung: nhiều kỹ thuật học không giám sát hơn (bao gồm phân cụm, phát hiện bất thường, ước lượng mật độ và mô hình hỗn hợp); nhiều kỹ thuật hơn để huấn luyện mạng nơ-ron sâu (bao gồm mạng tự chuẩn hóa); các kỹ thuật thị giác máy tính bổ sung (bao gồm Xception, SNet, phát hiện đối tượng với YOLO và phân đoạn ngữ nghĩa bằng R-CNN); xử lý chuỗi bằng mạng nơ-ron tích chập (CNN, bao gồm WaveNet); xử lý ngôn ngữ tự nhiên bằng mạng nơ-ron hồi quy (RNN), CNN và bộ chuyển đổi (Transformers); và GAN.
2. Giới thiệu thêm các thư viện và API (Keras, API Dữ liệu, TF-Agents cho Học tăng cường) cũng như huấn luyện và triển khai các mô hình TF ở quy mô lớn bằng cách sử dụng API Chiến lược Phân phối, TF-Serving và Nền tảng AI đám mây của Google. Đồng thời giới thiệu ngắn gọn về TF Transform, TFLite, TF Addons/Seq2Seq và TensorFlow.js.
3. Hãy thảo luận về một số kết quả quan trọng mới nhất từ nghiên cứu về Học sâu (Deep Learning).
4. Chuyển toàn bộ các chương về TensorFlow sang TensorFlow 2 và sử dụng cách triển khai của TensorFlow. Nên tham khảo API Keras (tf.keras) bất cứ khi nào có thể.
5. Cập nhật các ví dụ mã để sử dụng các phiên bản mới nhất của Scikit-Learn, NumPy, pandas, Matplotlib và các thư viện khác.
6. Làm rõ một số phần và sửa một số lỗi, nhờ vào rất nhiều phản hồi tuyệt vời. Từ độc giả.

Một số chương đã được thêm vào, một số chương khác được viết lại và một vài chương được sắp xếp lại thứ tự. Xem <https://homl.info/changes2> Để biết thêm chi tiết về những thay đổi trong phiên bản thứ hai.

Các nguồn tài liệu khác

Có rất nhiều nguồn tài liệu tuyệt vời để học về Học máy. Ví dụ, khóa học [Học máy của Andrew Ng trên Coursera](#). Thật tuyệt vời, mặc dù nó đòi hỏi một khoản đầu tư thời gian đáng kể (khoảng vài tháng).

Ngoài ra còn có rất nhiều trang web thú vị về Học máy, trong đó tất nhiên bao gồm cả [Hướng dẫn sử dụng xuất sắc của Scikit-Learn](#). Bạn cũng có thể thích [Dataquest](#), nơi cung cấp các bài hướng dẫn tương tác rất hay, và các blog về học máy như những blog được liệt kê trên [Quora](#). Cuối cùng, [trang web Deep Learning](#) có một danh sách tài nguyên hữu ích để tham khảo và học hỏi hơn.

Có rất nhiều sách nhập môn khác về Học máy. Cụ thể là:

- Cuốn sách "Khoa học dữ liệu từ đầu" của Joel Grus (O'Reilly) trình bày những kiến thức cơ bản về Học máy và triển khai một số thuật toán chính bằng Python thuần túy (từ đầu, như tên gọi cho thấy).
- Cuốn Machine Learning: An Algorithmic

Perspective của Stephen Marsland (Chapman & Hall) là một cuốn sách giới thiệu tuyệt vời về Học máy, bao gồm nhiều chủ đề chuyên sâu với các ví dụ mã bằng Python (cũng từ đầu, nhưng sử dụng NumPy).- Cuốn Python Machine Learning của Sebastian Raschka (Packt Publishing) cũng là một cuốn sách

giới thiệu tuyệt vời về Học máy và tận dụng các thư viện mã nguồn mở của Python (Pylearn 2 và Theano).

- Cuốn Deep Learning with Python của François Chollet (Manning) là một cuốn sách rất thực tế, bao gồm nhiều chủ đề một cách rõ

ràng và súc tích, như bạn mong đợi từ tác giả của thư viện Keras xuất sắc. Cuốn sách ưu tiên các ví dụ mã hơn là lý thuyết toán học.

- Cuốn sách "The Hundred-Page Machine Learning Book" của Andriy Burkov rất ngắn gọn nhưng bao quát một phạm vi chủ đề ấn tượng, giới thiệu chúng bằng ngôn ngữ dễ hiểu mà không hề né tránh các phương trình toán học.
- Cuốn sách Learning from Data (AMLBook) của Yaser S. Abu-Mostafa, Malik Magdon-Ismail và Hsuan-Tien Lin là một cách tiếp cận khả lý thuyết đối với học máy, cung cấp những hiểu biết sâu sắc, đặc biệt là về sự đánh đổi giữa độ lệch và phương sai (xem [Chương 4](#)).
- Cuốn sách "Trí tuệ nhân tạo: Một cách tiếp cận hiện đại", ấn bản thứ 3 (Pearson) của Stuart Russell và Peter Norvig là một cuốn sách tuyệt vời (và rất đồ sộ) bao quát một lượng lớn các chủ đề, trong đó có Học máy. Nó giúp chúng ta hiểu rõ hơn về Học máy.

Cuối cùng, tham gia các trang web tổ chức cuộc thi học máy như [Kaggle.com](#). Điều này sẽ cho phép bạn thực hành các kỹ năng của mình trên các vấn đề thực tế, với sự trợ giúp và những hiểu biết sâu sắc từ một số chuyên gia ML hàng đầu hiện nay.

Các quy ước được sử dụng trong cuốn sách này

Cuốn sách này sử dụng các quy ước in ấn sau:

Chữ

in nghiêng biểu thị các thuật ngữ, URL, địa chỉ email, tên tệp và phần mở rộng tệp mới.

Độ rộng cố định. Được

sử dụng cho danh sách chương trình, cũng như trong các đoạn văn để tham chiếu đến các thành phần của chương trình như tên biến hoặc hàm, cơ sở dữ liệu, kiểu dữ liệu, biến môi trường, câu lệnh và từ khóa.

Chữ đậm, độ rộng cố định.

Hiển thị các lệnh hoặc văn bản khác mà người dùng cần nhập chính xác.

Chữ nghiêng có độ rộng không đổi

Hiển thị văn bản cần được thay thế bằng các giá trị do người dùng cung cấp hoặc bằng các giá trị được xác định bởi ngữ cảnh.



Yếu tố này biểu thị một lời khuyên hoặc gợi ý.



Yếu tố này biểu thị một ghi chú chung.



Yếu tố này biểu thị một lời cảnh báo hoặc lưu ý.

Ví dụ mã

Có một loạt các sổ tay Jupyter chứa đầy tài liệu bổ sung, chẳng hạn như các ví dụ mã và bài tập, có sẵn để tải xuống tại <https://github.com/ageron/handson-ml2>.

Một số ví dụ mã trong sách đã lược bỏ các phần lặp lại hoặc các chi tiết hiển nhiên hoặc không liên quan đến Học máy. Điều này giúp tập trung vào các phần quan trọng của mã và tiết kiệm không gian để đề cập đến nhiều chủ đề hơn. Nếu bạn muốn xem toàn bộ ví dụ mã, chúng đều có sẵn trong các sổ tay Jupyter.

Lưu ý rằng khi các ví dụ mã hiển thị một số kết quả đầu ra, các ví dụ mã này được hiển thị với dấu nhắc Python (>>> và ...), giống như trong trình thông dịch Python, để phân biệt rõ ràng mã với kết quả đầu ra. Ví dụ, đoạn mã này định nghĩa hàm `square()`, sau đó nó tính toán và hiển thị bình phương của 3:

```
>>> def square(x):  
...     trả về x ** 2  
...  
  
>>> kết quả = bình phương(3) >>>  
kết quả  
9
```

Khi mã không hiển thị bất cứ thứ gì, các lời nhắc sẽ không được sử dụng. Tuy nhiên, đôi khi kết quả có thể được hiển thị dưới dạng chú thích, như thể này:

```
def square(x):
    trả về x ** 2

kết quả = bình phương(3) # kết quả là 9
```

Sử dụng ví dụ mã

Cuốn sách này được viết ra để giúp bạn hoàn thành công việc của mình. Nói chung, nếu sách này cung cấp mã ví dụ, bạn có thể sử dụng chúng trong chương trình và tài liệu của mình. Bạn không cần liên hệ với chúng tôi để xin phép trừ khi bạn sao chép một phần đáng kể mã nguồn. Ví dụ, việc viết một chương trình sử dụng nhiều đoạn mã từ cuốn sách này không cần xin phép. Việc bán hoặc phân phối đĩa CD-ROM chứa các ví dụ từ sách của O'Reilly thì cần xin phép. Việc trả lời câu hỏi bằng cách trích dẫn cuốn sách này và đưa ra mã ví dụ không cần xin phép. Việc kết hợp một lượng đáng kể mã ví dụ từ cuốn sách này vào tài liệu sản phẩm của bạn thì cần xin phép.

Chúng tôi đánh giá cao nhưng không bắt buộc việc ghi nguồn. Thông tin ghi nguồn thường bao gồm tiêu đề, tác giả, nhà xuất bản và ISBN. Ví dụ: "Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow, 2nd Edition, by Aurélien Géron (O'Reilly).

Bản quyền 2019 Kiwisoft SAS, 978-1-492-03264-9." Nếu bạn cảm thấy việc sử dụng các ví dụ mã nguồn của mình vượt quá phạm vi sử dụng hợp lý hoặc quyền được cho phép ở trên, vui lòng liên hệ với chúng tôi theo địa chỉ permissions@oreilly.com.

Học trực tuyến O'Reilly



Trong gần 40 năm, O'Reilly Media đã cung cấp đào tạo về công nghệ và kinh doanh, kiến thức và hiểu biết sâu sắc để giúp các công ty thành công.

Mạng lưới chuyên gia và nhà sáng tạo độc đáo của chúng tôi chia sẻ kiến thức và chuyên môn thông qua sách, bài báo, hội nghị và nền tảng học trực tuyến. Nền tảng học trực tuyến của O'Reilly cung cấp cho bạn quyền truy cập theo yêu cầu vào các khóa đào tạo trực tiếp, lộ trình học tập chuyên sâu, môi trường lập trình tương tác và một bộ sưu tập khổng lồ các tài liệu văn bản và video từ O'Reilly và hơn 200 nhà xuất bản khác. Để biết thêm thông tin, vui lòng truy cập <http://oreilly.com>.

Cách liên hệ với chúng tôi

Vui lòng gửi ý kiến và câu hỏi liên quan đến cuốn sách này đến nhà xuất bản:

Công ty truyền thông O'Reilly
1005 Gravenstein Highway North Sebastopol,
CA 95472 800-998-9938 (tại
Hoa Kỳ hoặc Canada) 707-829-0515 (quốc tế hoặc nội địa)
707-829-0104 (fax)

Chúng tôi có một trang web dành cho cuốn sách này, nơi chúng tôi liệt kê các lỗi sai, ví dụ và bất kỳ thông tin bổ sung nào khác. Bạn có thể truy cập trang này tại <https://homl.info/oreilly2>.

Để bình luận hoặc đặt câu hỏi kỹ thuật về cuốn sách này, vui lòng gửi email đến bookquestions@oreilly.com.

Để biết thêm thông tin về sách, khóa học, hội nghị và tin tức của chúng tôi, vui lòng truy cập trang web của chúng tôi tại <http://www.oreilly.com>.

Tìm chúng tôi trên Facebook: <http://facebook.com/oreilly> Theo

đổi chúng tôi trên Twitter: <http://twitter.com/oreillymedia>

Hãy xem chúng tôi trên YouTube: <http://www.youtube.com/oreillymedia>

Lời cảm ơn

Tôi chưa bao giờ tưởng tượng rằng ấn bản đầu tiên của cuốn sách này lại nhận được sự đón nhận lớn đến vậy. Tôi đã nhận được rất nhiều tin nhắn từ độc giả, nhiều người đặt câu hỏi, một số người tốt bụng chỉ ra lỗi sai, và hầu hết đều gửi cho tôi những lời động viên. Tôi không thể diễn tả hết lòng biết ơn của mình đối với tất cả các độc giả vì sự ủng hộ to lớn của họ.

Cảm ơn tất cả mọi người rất nhiều! Vui lòng dừng ngay [báo cáo sự cố trên GitHub](#). Nếu bạn phát hiện lỗi trong các ví dụ mã (hoặc chỉ để hỏi), hoặc để gửi [báo cáo lỗi](#), hãy liên hệ với chúng tôi. Nếu bạn phát hiện lỗi trong văn bản, hãy cho tôi biết. Một số độc giả cũng chia sẻ cách cuốn sách này giúp họ có được công việc đầu tiên, hoặc giúp họ giải quyết một vấn đề cụ thể mà họ đang gặp phải. Tôi thấy những phản hồi như vậy vô cùng tạo động lực. Nếu bạn thấy cuốn sách này hữu ích, tôi rất mong bạn có thể chia sẻ câu chuyện của mình với tôi, có thể là riêng tư (ví dụ: qua [LinkedIn](#)), hoặc công khai (ví dụ: trên Twitter hoặc qua [bài đánh giá trên Amazon](#)).

Tôi cũng vô cùng biết ơn tất cả những người tuyệt vời đã dành thời gian quý báu của mình để xem xét cuốn sách của tôi một cách kỹ lưỡng. Đặc biệt, tôi muốn cảm ơn François Chollet vì đã xem xét tất cả các chương liên quan đến Keras và TensorFlow và đã cho tôi những phản hồi sâu sắc tuyệt vời. Vì Keras là một trong những bổ sung chính cho ấn bản thứ hai này, việc tác giả của nó xem xét cuốn sách là vô cùng quý giá. Tôi đánh giá cao cuốn sách này.

Cuốn sách **Deep Learning with Python** của François (Manning): Nó có tính ngắn gọn, rõ ràng và chiều sâu như chính thư viện Keras. Xin gửi lời cảm ơn đặc biệt đến Ankur Patel, người đã xem xét từng chương của ấn bản thứ hai này và đưa ra những phản hồi tuyệt vời, đặc biệt là **Chương 9**, chương đề cập đến các kỹ thuật học không giám sát. Anh ấy có thể viết cả một cuốn sách về chủ đề này. Ồ, đợi đã, anh ấy đã viết rồi! Hãy xem cuốn "**Hands-On Unsupervised Learning Using Python: How to Build Applied Machine Learning Solutions from Unlabeled Data**" (O'Reilly). Xin gửi lời cảm ơn sâu sắc đến Olzhas Akpambetov, người đã xem xét tất cả các chương trong phần thứ hai của cuốn sách, kiểm tra phần lớn mã nguồn và đưa ra nhiều gợi ý tuyệt vời. Tôi cũng rất biết ơn Mark Daoust, Jon Krohn, Dominic Monn và Josh Patterson vì đã xem xét phần thứ hai của cuốn sách này một cách kỹ lưỡng và chia sẻ kiến thức chuyên môn của họ. Họ đã không bỏ sót bất cứ chi tiết nào và cung cấp những phản hồi vô cùng hữu ích.

Trong quá trình viết ấn bản thứ hai này, tôi đã rất may mắn nhận được nhiều sự giúp đỡ từ các thành viên của nhóm TensorFlow—đặc biệt là Martin Wicke, người đã không ngừng trả lời hàng tá câu hỏi của tôi và chuyển những câu hỏi còn lại đến đúng người, bao gồm Karmel Allison, Paige Bailey, Eugene Brevdo, William Chargin, Daniel “Wolff”.

Dobson, Nick Felt, Bruce Fontaine, Goldie Gadde, Sandeep Gupta, Priya Gupta, Kevin Haas, Konstantinos Katsiapis, Viacheslav Kovalevskiy, Allen Lavoie, Clemens Mewald, Dan Moldovan, Sean Morgan, Tom O'Malley, Alexandre Passos, André Susano Pinto, Anthony Platanios, Oscar Ramirez, Anna Revinskaya, Saurabh Saxena, Ryan Sepassi, Jiri Simsa, Xiaodan Song, Christina Sorokin, Dustin Tran, Todd Wang, Pete Warden (người cũng đã xem xét phiên bản đầu tiên), Edd Wilder-James và Yuefeng Zhou, tất cả đều đã giúp đỡ rất nhiều. Xin chân thành cảm ơn tất cả các bạn, và tất cả các thành viên khác của nhóm TensorFlow, không chỉ vì sự giúp đỡ mà còn vì đã tạo ra một thư viện tuyệt vời như vậy! Đặc biệt cảm ơn Irene Giannoumis và Robert Crowe của nhóm TFX đã xem xét kỹ lưỡng **Chương 13** và **19**.

Tôi cũng xin gửi lời cảm ơn chân thành đến đội ngũ nhân viên tuyệt vời của O'Reilly, đặc biệt là Nicole Taché, người đã cho tôi những phản hồi sâu sắc và luôn vui vẻ, động viên và giúp đỡ: Tôi không thể mơ ước có một biên tập viên nào tốt hơn. Cảm ơn rất nhiều Michele Cronin, người đã rất hữu ích (và kiên nhẫn) trong giai đoạn đầu của ấn bản thứ hai này, và Kristen Brown, biên tập viên sản xuất cho ấn bản thứ hai, người đã giám sát toàn bộ quá trình (cô ấy cũng điều phối các bản sửa lỗi và cập nhật cho mỗi lần tái bản của ấn bản đầu tiên). Cảm ơn Rachel Mon-aghan và Amanda Kersey vì đã biên tập bản thảo kỹ lưỡng (tương ứng cho ấn bản đầu tiên và thứ hai), và Johnny O'Toole, người đã quản lý mối quan hệ với Amazon và trả lời nhiều câu hỏi của tôi. Cảm ơn Marie Beaugureau, Ben Lorica, Mike Loukides và Laurel Ruma vì đã tin tưởng vào dự án này và giúp tôi xác định phạm vi của nó. Xin cảm ơn Matt Hacker và toàn thể nhóm Atlas đã giải đáp mọi thắc mắc kỹ thuật của tôi về định dạng, AsciiDoc và LaTeX, và cảm ơn Nick Adams, Rebecca Demarest, Rachel Head, Judith McConville, Helen Monroe, Karen Montgomery, Rachel Roumeliotis và tất cả những người khác tại O'Reilly đã đóng góp cho cuốn sách này.

Tôi cũng muốn gửi lời cảm ơn đến các đồng nghiệp cũ tại Google, đặc biệt là nhóm phân loại video YouTube, vì đã dạy tôi rất nhiều về Học máy. Tôi không thể nào bắt đầu phiên bản đầu tiên nếu không có họ. Xin gửi lời cảm ơn đặc biệt đến những người thầy về Học máy của tôi: Clément Courbet, Julien Dubois, Mathias Kende, Daniel Kitachewsky, James Pack, Alexander Pak, Anosh Raj, Vitor Sessak, Wiktor Tomczak, Ingrid von Glehn và Rich Washington. Và cảm ơn tất cả những người khác mà tôi đã làm việc cùng tại YouTube và trong các nhóm nghiên cứu tuyệt vời của Google ở Mountain View. Xin chân thành cảm ơn Martin Andrews, Sam Witteveen và Jason Zaman vì đã chào đón tôi vào nhóm Chuyên gia Phát triển Google tại Singapore, với sự hỗ trợ tận tình của Soonson Kwon, và vì tất cả những cuộc thảo luận tuyệt vời mà chúng ta đã có về Học sâu và TensorFlow. Bất cứ ai quan tâm đến Học sâu tại Singapore chắc chắn nên tham gia buổi gặp mặt **Deep Learning Singapore** của họ. Jason xứng đáng nhận được lời cảm ơn đặc biệt vì đã chia sẻ một số kiến thức chuyên môn về TFLite của mình cho Chương 19!

Tôi sẽ không bao giờ quên những người tốt bụng đã đánh giá ấn bản đầu tiên của cuốn sách này, bao gồm David Andrzejewski, Lukas Biewald, Justin Francis, Vincent Guilbeau, Eddy Hung, Karim Matrah, Grégoire Mesnil, Salim Sémaoune, Iain Smears, Michel Tessier, Ingrid von Glehn, Pete Warden, và tất nhiên là người anh trai thân yêu của tôi, Sylvain. Đặc biệt cảm ơn Haesun Park, người đã cho tôi rất nhiều phản hồi tuyệt vời và phát hiện ra một số lỗi trong khi anh ấy dịch sang tiếng Hàn cho ấn bản đầu tiên của cuốn sách này. Anh ấy cũng đã dịch các notebook Jupyter sang tiếng Hàn, chưa kể đến tài liệu của TensorFlow. Tôi không nói tiếng Hàn, nhưng xét theo chất lượng phản hồi của anh ấy, tất cả các bản dịch của anh ấy chắc chắn phải rất xuất sắc! Haesun cũng đã đóng góp một số lời giải cho các bài tập trong ấn bản thứ hai này.

Cuối cùng nhưng không kém phần quan trọng, tôi vô cùng biết ơn người vợ yêu quý của mình, Emmanuelle, và ba đứa con tuyệt vời của chúng tôi, Alexandre, Rémi và Gabrielle, vì đã động viên tôi nỗ lực hết mình cho cuốn sách này. Tôi cũng cảm ơn sự tò mò không ngừng nghỉ của các con: việc giải thích một số khái niệm khó hiểu nhất trong cuốn sách này cho vợ con đã giúp tôi làm rõ suy nghĩ và trực tiếp cải thiện nhiều phần của nó. Và các con vẫn luôn mang bánh quy và cà phê đến cho tôi! Còn gì tuyệt vời hơn nữa?

PHẦN I

Những nguyên tắc cơ bản của

Học máy

CHƯƠNG 1

Bức tranh tổng quan về học máy

Khi hầu hết mọi người nghe đến "Học máy" (Machine Learning), họ hình dung ra một con robot: một người quản gia đáng tin cậy hoặc một Kẻ hủy diệt nguy hiểm, tùy thuộc vào người bạn hỏi. Nhưng Học máy không chỉ là một viễn tưởng tương lai; nó đã hiện diện rồi. Trên thực tế, nó đã tồn tại hàng thập kỷ trong một số ứng dụng chuyên biệt, chẳng hạn như Nhận dạng ký tự quang học (OCR). Nhưng ứng dụng Học máy đầu tiên thực sự trở nên phổ biến, cải thiện cuộc sống của hàng trăm triệu người, đã chiếm lĩnh thế giới vào những năm 1990: bộ lọc thư rác. Nó không hẳn là một Skynet tự nhận thức, nhưng về mặt kỹ thuật, nó đủ điều kiện là Học máy (nó thực sự đã học tốt đến mức bạn hiếm khi cần phải đánh dấu email là thư rác nữa). Sau đó là hàng trăm ứng dụng Học máy hiện đang âm thầm hỗ trợ hàng trăm sản phẩm và tính năng mà bạn sử dụng thường xuyên, từ đề xuất tốt hơn đến tìm kiếm bằng giọng nói.

Máy học bắt đầu từ đâu và kết thúc ở đâu? Chính xác thì việc một cỗ máy học được điều gì đó có nghĩa là gì? Nếu tôi tải xuống một bản sao của Wikipedia, liệu máy tính của tôi có thực sự học được điều gì đó không? Liệu nó có đột nhiên thông minh hơn không? Trong chương này, chúng ta sẽ bắt đầu bằng việc làm rõ Máy học là gì và tại sao bạn có thể muốn sử dụng nó.

Trước khi bắt đầu khám phá "lục địa" Học máy, chúng ta sẽ xem xét bản đồ và tìm hiểu về các khu vực chính cũng như những địa danh nổi bật: học có giám sát so với học không giám sát, học trực tuyến so với học theo lô, học dựa trên trường hợp so với học dựa trên mô hình. Sau đó, chúng ta sẽ xem xét quy trình làm việc của một dự án Học máy điển hình, thảo luận về những thách thức chính mà bạn có thể gặp phải, và tìm hiểu cách đánh giá và tinh chỉnh một hệ thống Học máy.

Chương này giới thiệu rất nhiều khái niệm cơ bản (và thuật ngữ chuyên ngành) mà mọi nhà khoa học dữ liệu nên nắm vững. Đây sẽ là một cái nhìn tổng quan cấp cao (đây là chương duy nhất không có nhiều mã lập trình), tất cả đều khá đơn giản, nhưng bạn nên đảm bảo mọi thứ đều rõ ràng trước khi tiếp tục đọc các phần còn lại của cuốn sách. Vậy thì hãy pha một tách cà phê và cùng bắt đầu nào!



Nếu bạn đã nắm vững các kiến thức cơ bản về Học máy, bạn có thể bỏ qua thẳng đến **Chương 2**. Nếu bạn chưa chắc chắn, hãy cố gắng trả lời tất cả các câu hỏi ở cuối chương trước khi chuyển sang chương tiếp theo.

Máy học là gì?

Máy học là khoa học (và nghệ thuật) lập trình máy tính để chúng có thể học hỏi từ dữ liệu.

Dưới đây là một định nghĩa tổng quát hơn một chút:

[Máy học] là lĩnh vực nghiên cứu giúp máy tính có khả năng học hỏi mà không cần được lập trình rõ ràng.

—Arthur Samuel, 1959

Và một phiên bản thiên về kỹ thuật hơn:

Một chương trình máy tính được cho là học hỏi từ kinh nghiệm E liên quan đến một nhiệm vụ T và một thước đo hiệu suất P, nếu hiệu suất của nó trên nhiệm vụ T, được đo bằng P, cải thiện theo kinh nghiệm E.

—Tom Mitchell, 1997

Bộ lọc thư rác của bạn là một chương trình Học máy, khi được cung cấp các ví dụ về email rác (ví dụ: được người dùng gắn cờ) và các ví dụ về email thông thường (không phải thư rác, còn được gọi là "ham"), có thể học cách gắn cờ thư rác. Các ví dụ mà hệ thống sử dụng để học được gọi là tập huấn luyện. Mỗi ví dụ huấn luyện được gọi là một trường hợp huấn luyện (hoặc mẫu). Trong trường hợp này, nhiệm vụ T là gắn cờ thư rác cho các email mới, kinh nghiệm E là dữ liệu huấn luyện và thước đo hiệu suất P cần được xác định; ví dụ, bạn có thể sử dụng tỷ lệ email được phân loại chính xác. Thước đo hiệu suất cụ thể này được gọi là độ chính xác và nó thường được sử dụng trong các nhiệm vụ phân loại.

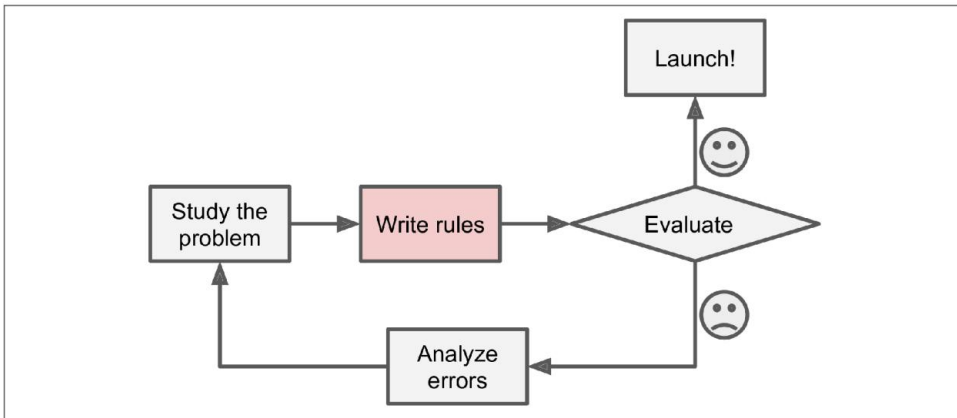
Nếu bạn chỉ tải xuống một bản sao của Wikipedia, máy tính của bạn sẽ có nhiều dữ liệu hơn, nhưng nó không đột nhiên hoạt động tốt hơn trong bất kỳ tác vụ nào. Do đó, việc tải xuống một bản sao của Wikipedia không phải là Học máy.

Tại sao nên sử dụng máy học?

Hãy xem xét cách bạn sẽ viết một bộ lọc thư rác bằng các kỹ thuật lập trình truyền thống (**Hình 1-1**):

1. Đầu tiên, bạn cần xem xét thư rác thường trông như thế nào. Bạn có thể nhận thấy một số từ hoặc cụm từ (như "4U," "thề tìn dụng," "miễn phí," và "tuyệt vời") thường xuất hiện nhiều trong tiêu đề thư. Có lẽ bạn cũng sẽ nhận thấy một vài mẫu khác trong tên người gửi, nội dung email và các phần khác của email.

2. Bạn sẽ viết một thuật toán phát hiện cho mỗi mẫu mà bạn nhận thấy, và chương trình của bạn sẽ gắn cờ các email là thư rác nếu phát hiện thấy một số lượng nhất định các mẫu này.
3. Bạn sẽ kiểm tra chương trình của mình và lặp lại bước 1 và 2 cho đến khi nó đạt chất lượng tốt để khởi động.



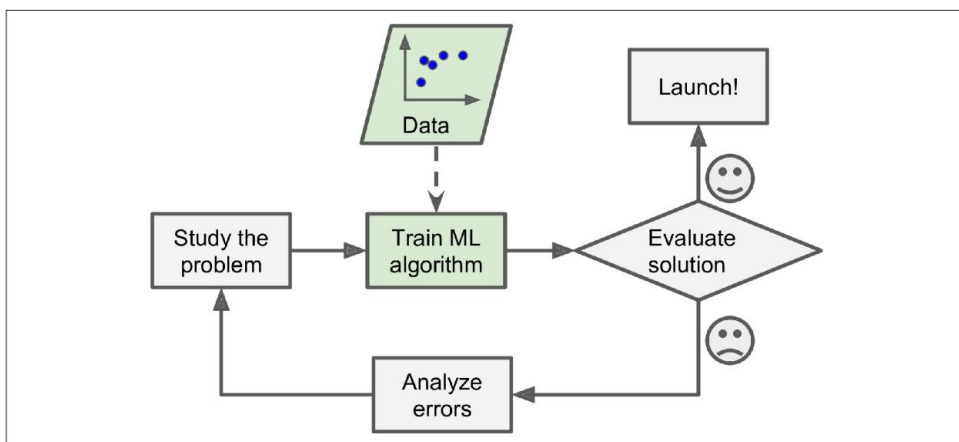
Hình 1-1. Phương pháp truyền thống

Vì bài toán khó, chương trình của bạn có thể sẽ trở thành một danh sách dài các quy tắc phức tạp – rất khó bảo trì.

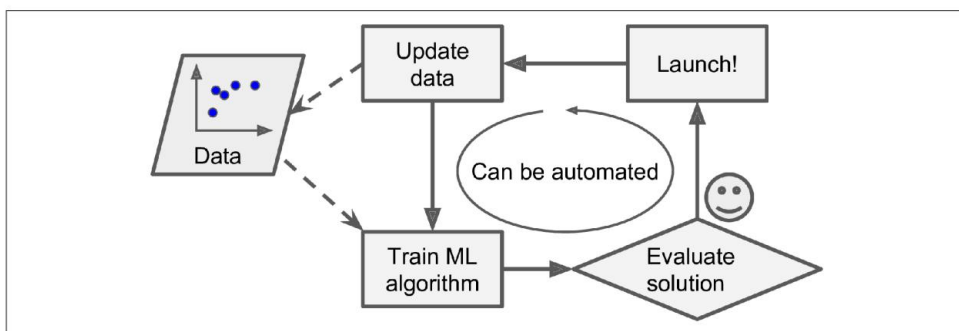
Ngược lại, bộ lọc thư rác dựa trên kỹ thuật Học máy tự động học cách nhận biết những từ và cụm từ nào là yếu tố dự đoán thư rác tốt bằng cách phát hiện các mẫu từ xuất hiện thường xuyên bất thường trong các ví dụ thư rác so với các ví dụ thư hợp lệ (Hình 1-2). Chương trình ngắn gọn hơn nhiều, dễ bảo trì hơn và có nhiều khả năng hiệu quả hơn chính xác.

Điều gì sẽ xảy ra nếu những kẻ gửi thư rác nhận thấy tất cả email của chúng chứa "4U" đều bị chặn? Chúng có thể bắt đầu viết "For U" thay thế. Một bộ lọc thư rác sử dụng các kỹ thuật lập trình truyền thống sẽ cần được cập nhật để gắn cờ các email "For U". Nếu những kẻ gửi thư rác tiếp tục tìm cách vượt qua bộ lọc thư rác của bạn, bạn sẽ cần phải liên tục viết các quy tắc mới.

Ngược lại, bộ lọc thư rác dựa trên các kỹ thuật Học máy sẽ tự động nhận thấy rằng cụm từ "For U" xuất hiện với tần suất bất thường trong thư rác do người dùng báo cáo và bắt đầu gắn cờ chúng mà không cần sự can thiệp của bạn (Hình 1-3).



Hình 1-2. Phương pháp học máy

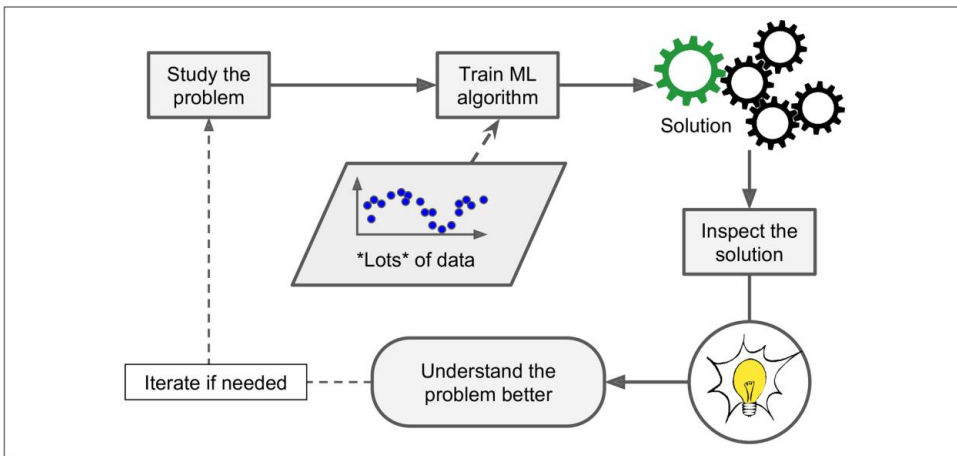


Hình 1-3. Tự động thích ứng với sự thay đổi

Một lĩnh vực khác mà Học máy tỏa sáng là giải quyết các vấn đề quá phức tạp đối với các phương pháp truyền thống hoặc không có thuật toán nào được biết đến. Ví dụ, hãy xem xét nhận dạng giọng nói. Giả sử bạn muốn bắt đầu đơn giản và viết một chương trình có khả năng phân biệt các từ "một" và "hai". Bạn có thể nhận thấy rằng từ "hai" bắt đầu bằng một âm cao ("T"), vì vậy bạn có thể lập trình sẵn một thuật toán đo cường độ âm cao và sử dụng nó để phân biệt một và hai - nhưng rõ ràng kỹ thuật này sẽ không thể mở rộng quy mô cho hàng nghìn từ được nói bởi hàng triệu người rất khác nhau trong môi trường ồn ào và bằng hàng chục ngôn ngữ. Giải pháp tốt nhất (ít nhất là hiện nay) là viết một thuật toán tự học, dựa trên nhiều bản ghi ví dụ cho mỗi từ.

Cuối cùng, Học máy có thể giúp con người học hỏi (Hình 1-4). Các thuật toán Học máy có thể được kiểm tra để xem chúng đã học được gì (mặc dù đối với một số thuật toán thì điều này có thể khó khăn). Ví dụ, sau khi bộ lọc thư rác được huấn luyện với đủ lượng thư rác, nó có thể dễ dàng được kiểm tra để tiết lộ danh sách các từ và sự kết hợp các từ mà nó cho là những yếu tố dự đoán thư rác tốt nhất. Đôi khi điều này sẽ tiết lộ những điều không ngờ tới.

Các mối tương quan hoặc xu hướng mới có thể giúp phát hiện ra các mô hình mà ban đầu không dễ nhận thấy. Áp dụng các kỹ thuật học máy để phân tích lượng lớn dữ liệu có thể giúp khám phá ra những mô hình không dễ nhận thấy ngay lập tức. Điều này được gọi là khai thác dữ liệu.



Hình 1-4. Máy học có thể giúp con người học hỏi.

Tóm lại, Học máy rất hữu ích cho:

- Những vấn đề mà các giải pháp hiện có đòi hỏi nhiều tinh chỉnh hoặc danh sách dài các quy tắc: một thuật toán Học máy thường có thể đơn giản hóa mã và hoạt động tốt hơn so với phương pháp truyền thống.
- Các vấn đề phức tạp mà phương pháp truyền thống không mang lại giải pháp hiệu quả: các kỹ thuật Học máy tốt nhất có thể tìm ra giải pháp.
- Môi trường biến động: hệ thống

Học máy có thể thích ứng với dữ liệu mới. • Thu thập thông tin chi tiết về các vấn đề phức tạp và lượng dữ liệu lớn.

Ví dụ về các ứng dụng

Hãy cùng xem xét một số ví dụ cụ thể về các nhiệm vụ Học máy, cùng với các kỹ thuật có thể giải quyết chúng:

Phân tích hình ảnh sản phẩm trên dây chuyền sản xuất để tự động phân loại chúng.

Đây là phân loại hình ảnh, thường được thực hiện bằng cách sử dụng mạng nơ-ron tích chập (CNN; xem [Chương 14](#)).

Phát hiện khối u trong ảnh chụp

não. Đây là phân đoạn ngữ nghĩa, trong đó mỗi pixel trong ảnh được phân loại (vì chúng ta muốn xác định vị trí và hình dạng chính xác của khối u), thường cũng sử dụng mạng nơ-ron tích chập (CNN).

Tự động phân loại bài báo

Đây là xử lý ngôn ngữ tự nhiên (NLP), và cụ thể hơn là phân loại văn bản, có thể được giải quyết bằng cách sử dụng mạng nơ-ron hồi quy (RNN), mạng nơ-ron tích chập (CNN) hoặc Transformer (xem [Chương 16](#)).

Tự động gắn cờ các bình luận xúc phạm trên diễn đàn thảo luận. Đây cũng là

phân loại văn bản, sử dụng cùng các công cụ xử lý ngôn ngữ tự nhiên (NLP).

Tóm tắt tài liệu dài một cách tự động. Đây là một

nhánh của xử lý ngôn ngữ tự nhiên (NLP) được gọi là tóm tắt văn bản, cũng sử dụng các công cụ tương tự.

Tạo chatbot hoặc trợ lý cá nhân

Điều này bao gồm nhiều thành phần xử lý ngôn ngữ tự nhiên (NLP), trong đó có các mô-đun hiểu ngôn ngữ tự nhiên (NLU) và trả lời câu hỏi.

Dự báo doanh thu của công ty bạn trong năm tới, dựa trên nhiều chỉ số hiệu suất.

Đây là một bài toán hồi quy (tức là dự đoán giá trị) có thể được giải quyết bằng bất kỳ mô hình hồi quy nào, chẳng hạn như mô hình hồi quy tuyến tính hoặc hồi quy đa thức (xem [Chương 4](#)), SVM hồi quy (xem [Chương 5](#)), rừng ngẫu nhiên hồi quy (xem [Chương 7](#)) hoặc mạng nơ-ron nhân tạo (xem [Chương 10](#)). Nếu bạn muốn tính đến chuỗi các chỉ số hiệu suất trong quá khứ, bạn có thể muốn sử dụng RNN, CNN hoặc Transformer (xem [Chương 15](#) và [16](#)).

Giúp ứng dụng của bạn phản hồi lệnh thoại

Đây là công nghệ nhận dạng giọng nói, đòi hỏi xử lý các mẫu âm thanh: vì chúng là các chuỗi dài và phức tạp, nên chúng thường được xử lý bằng RNN, CNN hoặc Transformer (xem [Chương 15](#) và [16](#)).

Phát hiện gian lận thẻ tín dụng.

Đây là phát hiện bất thường (xem [Chương 9](#)).

Phân khúc khách hàng dựa trên hành vi mua hàng của họ để bạn có thể thiết kế chiến lược tiếp thị khác nhau cho từng phân khúc. Đây được

gọi là phân cụm (xem [Chương 9](#)).

Biểu diễn một tập dữ liệu phức tạp, đa chiều thành một sơ đồ rõ ràng và dễ hiểu.

Đây là hình thức trực quan hóa dữ liệu, thường liên quan đến các kỹ thuật giảm chiều dữ liệu (xem [Chương 8](#)).

Đề xuất sản phẩm mà khách hàng có thể quan tâm, dựa trên lịch sử mua hàng trước đó.

Đây là một hệ thống đề xuất. Một cách tiếp cận là đưa dữ liệu về các giao dịch mua hàng trước đây (và các thông tin khác về khách hàng) vào mạng nơ-ron nhân tạo (xem [Chương...](#)).

(**ter** 10), và yêu cầu nó đưa ra dự đoán về lần mua hàng tiếp theo có khả năng xảy ra nhất. Mạng nơ-ron này thường được huấn luyện dựa trên chuỗi các giao dịch mua hàng trong quá khứ của tất cả khách hàng.

Xây dựng một bot thông minh cho trò chơi

Vấn đề này thường được giải quyết bằng cách sử dụng Học tăng cường (Reinforcement Learning - RL; xem **Chương** 18), một nhánh của Học máy (Machine Learning) huấn luyện các tác nhân (như bot) để chọn các hành động nhằm tối đa hóa phần thưởng của chúng theo thời gian (ví dụ: bot có thể nhận được phần thưởng mỗi khi người chơi mất điểm sinh mệnh), trong một môi trường nhất định (như trò chơi). Chương trình AlphaGo nổi tiếng đã đánh bại nhà vô địch thế giới cờ vây được xây dựng bằng RL.

Danh sách này có thể kéo dài mãi, nhưng hy vọng nó sẽ giúp bạn hình dung được phạm vi và độ phức tạp đáng kinh ngạc của các nhiệm vụ mà Học máy có thể giải quyết, cũng như các loại kỹ thuật bạn sẽ sử dụng cho mỗi nhiệm vụ.

Các loại hệ thống học máy

Có rất nhiều loại hệ thống Học máy khác nhau, vì vậy việc phân loại chúng thành các nhóm lớn dựa trên các tiêu chí sau đây là rất hữu ích:

- Liệu chúng có được huấn luyện dưới sự giám sát của con người hay không (có giám sát, không giám sát, bán giám sát và Học tăng cường)
- Liệu chúng có thể học

tăng dần trong quá trình thực hiện hay không (trực tuyến so với theo lô)
học hỏi)

- Cho dù chúng hoạt động bằng cách đơn giản so sánh các điểm dữ liệu mới với các điểm dữ liệu đã biết, hay bằng cách phát hiện các mẫu trong dữ liệu huấn luyện và xây dựng mô hình dự đoán, tương tự như cách các nhà khoa học làm (học dựa trên trường hợp so với học dựa trên mô hình)

Các tiêu chí này không loại trừ lẫn nhau; bạn có thể kết hợp chúng theo bất kỳ cách nào bạn muốn. Ví dụ, một bộ lọc thư rác hiện đại có thể học hỏi tức thời bằng cách sử dụng mô hình mạng nơ-ron sâu được huấn luyện bằng các ví dụ về thư rác và thư hợp lệ; điều này biến nó thành một hệ thống học tập có giám sát, dựa trên mô hình, trực tuyến.

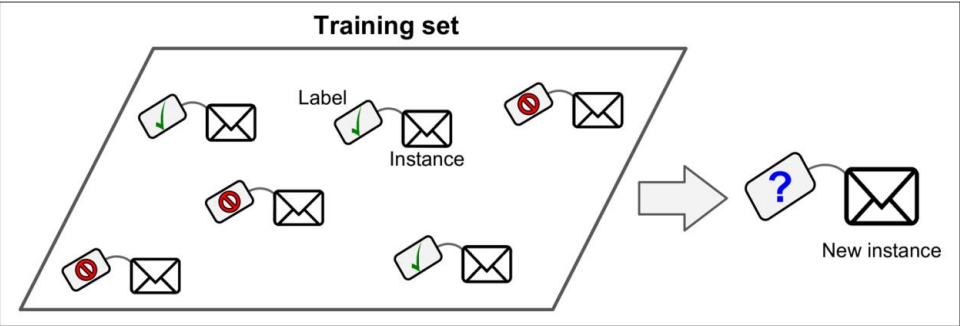
Chúng ta hãy cùng xem xét kỹ hơn từng tiêu chí này.

Các hệ thống học máy có thể được phân loại theo

mức độ và loại hình giám sát mà chúng nhận được trong quá trình huấn luyện. Có bốn loại chính: học có giám sát, học không giám sát, học bán giám sát và học tăng cường.

Học có giám sát Trong

học có giám sát, tập dữ liệu huấn luyện mà bạn cung cấp cho thuật toán bao gồm các giải pháp mong muốn, được gọi là nhãn (Hình 1-5).



Hình 1-5. Tập dữ liệu huấn luyện có nhãn để phân loại thư rác (một ví dụ về học có giám sát)

Phân loại là một nhiệm vụ học có giám sát điển hình. Bộ lọc thư rác là một ví dụ điển hình: nó được huấn luyện với nhiều email ví dụ cùng với loại thư của chúng (thư rác hoặc thư hợp lệ), và nó phải học cách phân loại các email mới.

Một nhiệm vụ điển hình khác là dự đoán một giá trị số mục tiêu, chẳng hạn như giá của một chiếc ô tô, dựa trên một tập hợp các đặc điểm (số dặm đã đi, tuổi đời, thương hiệu, v.v.) được gọi là các biến dự đoán. Loại nhiệm vụ này được gọi là hồi quy (Hình 1-6).¹ Để huấn luyện hệ thống, bạn cần cung cấp cho nó nhiều ví dụ về ô tô, bao gồm cả các biến dự đoán và nhãn của chúng (tức là giá của chúng).



Trong Học máy, thuộc tính là một kiểu dữ liệu (ví dụ: “số dặm”), trong khi đặc trưng có nhiều nghĩa khác nhau, tùy thuộc vào ngữ cảnh, nhưng nhìn chung có nghĩa là một thuộc tính cộng với giá trị của nó (ví dụ: “số dặm = 15.000”). Nhiều người sử dụng các từ thuộc tính và đặc trưng thay thế cho nhau.

Lưu ý rằng một số thuật toán hồi quy cũng có thể được sử dụng cho phân loại, và ngược lại. Ví dụ, hồi quy logistic thường được sử dụng cho phân loại, vì nó có thể đưa ra giá trị tương ứng với xác suất thuộc về một lớp nhất định (ví dụ: 20% khả năng là thư rác).

1. Một sự thật thú vị: Cái tên nghe có vẻ lạ này là một thuật ngữ thống kê do Francis Galton đưa ra khi ông nghiên cứu hiện tượng con cái của những người cao thường thấp hơn cha mẹ. Vì con cái của họ thấp hơn, ông gọi hiện tượng này là sự hồi quy về giá trị trung bình. Sau đó, thuật ngữ này được áp dụng cho các phương pháp ông sử dụng để phân tích mối tương quan giữa các biến số.



Hình 1-6. Bài toán hồi quy: dự đoán một giá trị, dựa trên một đặc trưng đầu vào (thường có nhiều đặc trưng đầu vào, và đôi khi có nhiều giá trị đầu ra)

Dưới đây là một số thuật toán học có giám sát quan trọng nhất (được đề cập trong cuốn sách này):

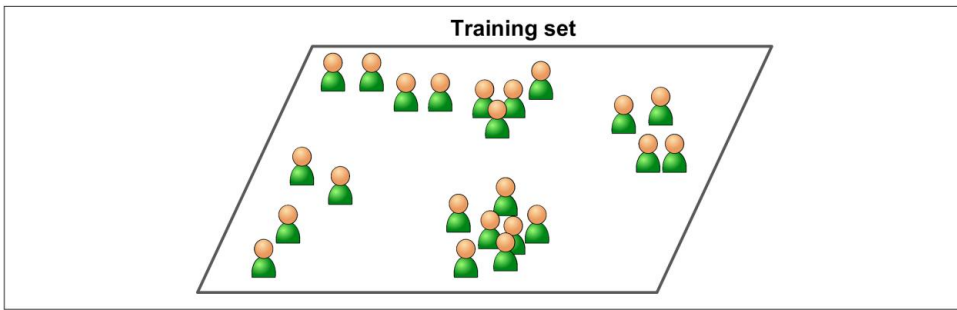
- Thuật toán k-hàng xóm gần nhất
- Hồi quy tuyến tính
- Hồi quy logistic
- Máy vectơ hỗ trợ (SVM)
- Cây quyết định và rừng ngẫu nhiên
- Mạng thần kinh2

Học không giám sát:

Trong học không giám sát, như bạn có thể đoán, dữ liệu huấn luyện không được gán nhãn (Hình 1-7).

Hệ thống cố gắng học mà không cần người hướng dẫn.

2. Một số kiến trúc mạng nơ-ron có thể là không giám sát, chẳng hạn như bộ mã hóa tự động và máy Boltzmann hạn chế. Chúng cũng có thể là bán giám sát, chẳng hạn như trong mạng niềm tin sâu và huấn luyện trước không giám sát.

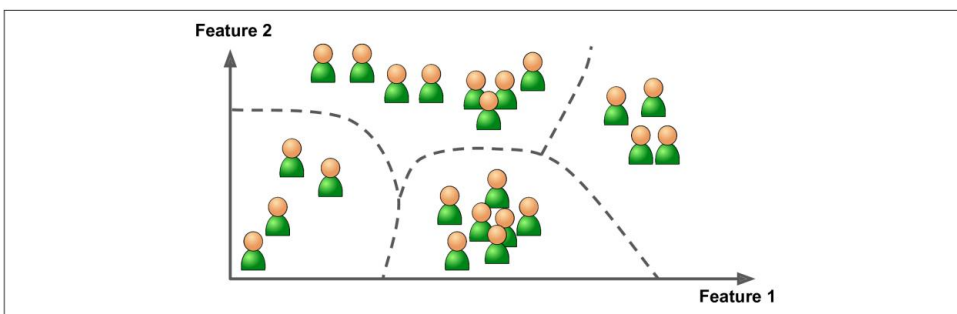


Hình 1-7. Tập dữ liệu huấn luyện không nhãn cho học không giám sát.

Dưới đây là một số thuật toán học không giám sát quan trọng nhất (hầu hết các thuật toán này đều được đề cập trong Chương 8 và 9):

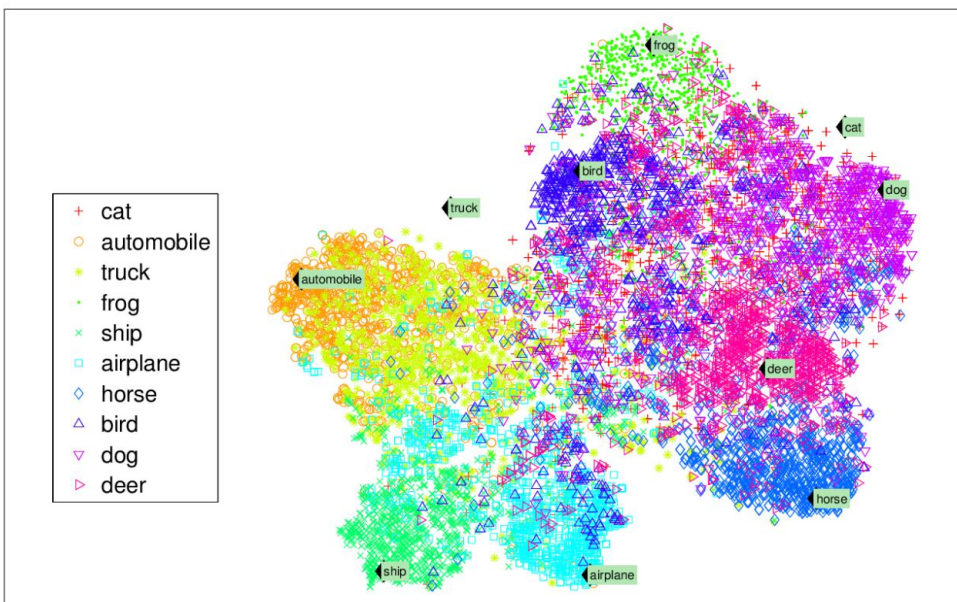
- Phân cụm
 - K-Means
 - DBSCAN
 - Phân tích cụm phân cấp (HCA) • Phát
- hiện bất thường và phát hiện tính mới
 - SVM một lớp
 - Rừng Cỗ Lập
- Trực quan hóa và giảm chiều dữ liệu
 - Phân tích thành phần chính (PCA)
 - PCA hạt nhân
 - Phương pháp nhúng tuyến tính cục bộ (LLE) – Phương pháp nhúng lân cận ngẫu nhiên phân bố t (t-SNE)
- Học luật liên kết
 - Tiên nghiệm
 - Eclat

Ví dụ, giả sử bạn có rất nhiều dữ liệu về khách truy cập blog của mình. Bạn có thể muốn chạy thuật toán phân cụm để cố gắng phát hiện các nhóm khách truy cập tương tự nhau (Hình 1-8). Bạn không cần phải cho thuật toán biết khách truy cập thuộc nhóm nào: nó sẽ tự tìm ra các mối liên hệ đó mà không cần sự trợ giúp của bạn. Ví dụ, nó có thể nhận thấy rằng 40% khách truy cập của bạn là nam giới yêu thích truyện tranh và thường đọc blog của bạn vào buổi tối, trong khi 20% là những người trẻ tuổi yêu thích khoa học viễn tưởng và truy cập vào cuối tuần. Nếu bạn sử dụng thuật toán phân cụm phân cấp, nó cũng có thể chia nhỏ mỗi nhóm thành các nhóm nhỏ hơn. Điều này có thể giúp bạn nhắm mục tiêu bài đăng của mình cho từng nhóm.



Hình 1-8. Phân cụm

Các thuật toán trực quan hóa cũng là những ví dụ điển hình của thuật toán học không giám sát: bạn cung cấp cho chúng rất nhiều dữ liệu phức tạp và chưa được gắn nhãn, và chúng sẽ xuất ra một biểu diễn 2D hoặc 3D của dữ liệu mà bạn có thể dễ dàng vẽ đồ thị (Hình 1-9). Các thuật toán này cố gắng bảo toàn cấu trúc càng nhiều càng tốt (ví dụ: cố gắng giữ cho các cụm riêng biệt trong không gian đầu vào không bị chồng chéo trong hình ảnh trực quan) để bạn có thể hiểu cách dữ liệu được tổ chức và có thể xác định các mẫu không ngờ tới.

Hình 1-9. Ví dụ về hình ảnh trực quan t-SNE làm nổi bật các cụm ngữ nghĩa³

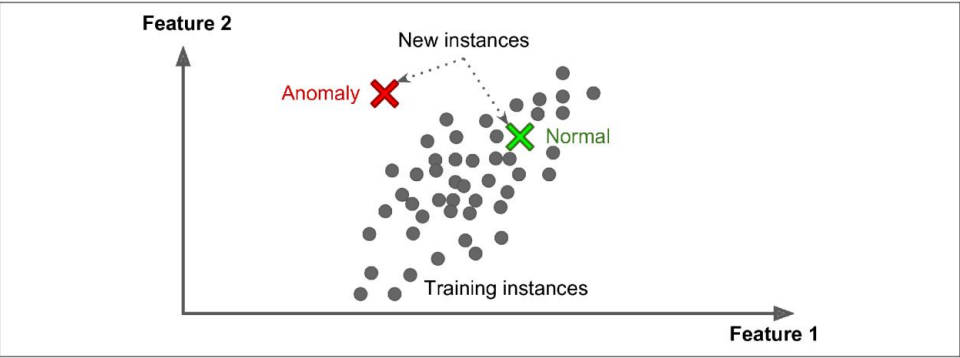
3. Hãy chú ý cách các loài động vật được tách biệt khá rõ ràng khỏi các phương tiện giao thông và cách ngựa ở gần hươu nhưng lại xa chim. Hình ảnh được sao chép với sự cho phép từ Richard Socher và cộng sự, "Học tập không cần dữ liệu huấn luyện thông qua chuyển giao đa phương thức", Kỷ yếu Hội nghị quốc tế lần thứ 26 về Hệ thống xử lý thông tin thần kinh 1 (2013): 935-943.

Một nhiệm vụ liên quan là giảm chiều dữ liệu, trong đó mục tiêu là đơn giản hóa dữ liệu mà không làm mất quá nhiều thông tin. Một cách để làm điều này là hợp nhất một số đặc trưng tương quan thành một. Ví dụ, số km đã đi của một chiếc xe có thể tương quan chặt chẽ với tuổi đời của nó, vì vậy thuật toán giảm chiều dữ liệu sẽ hợp nhất chúng thành một đặc trưng đại diện cho sự hao mòn của xe. Điều này được gọi là trích xuất đặc trưng.



Thường thì việc giảm kích thước dữ liệu huấn luyện bằng thuật toán giảm chiều trước khi đưa vào thuật toán Học máy khác (như thuật toán học có giám sát) là một ý tưởng hay. Quá trình này sẽ giúp thuật toán chạy nhanh hơn, dữ liệu chiếm ít dung lượng ổ đĩa và bộ nhớ hơn, và trong một số trường hợp, hiệu suất cũng có thể tốt hơn.

Một nhiệm vụ quan trọng khác không cần giám sát là phát hiện bất thường—ví dụ, phát hiện các giao dịch thẻ tín dụng bất thường để ngăn chặn gian lận, phát hiện lỗi sản xuất hoặc tự động loại bỏ các giá trị ngoại lai khỏi tập dữ liệu trước khi đưa vào thuật toán học khác. Hệ thống được cho xem chủ yếu các trường hợp bình thường trong quá trình huấn luyện, vì vậy nó học cách nhận biết chúng; sau đó, khi thấy một trường hợp mới, nó có thể cho biết liệu trường hợp đó trông giống như một trường hợp bình thường hay có khả năng là một bất thường (xem Hình 1-10). Một nhiệm vụ rất tương tự là phát hiện sự mới lạ: nó nhằm mục đích phát hiện các trường hợp mới trông khác biệt so với tất cả các trường hợp trong tập huấn luyện. Điều này đòi hỏi phải có một tập huấn luyện rất "sạch", không có bất kỳ trường hợp nào mà bạn muốn thuật toán phát hiện. Ví dụ, nếu bạn có hàng nghìn bức ảnh về chó, và 1% trong số những bức ảnh này là chó Chihuahua, thì thuật toán phát hiện sự mới lạ không nên coi những bức ảnh mới về chó Chihuahua là sự mới lạ. Mặt khác, các thuật toán phát hiện bất thường có thể coi những con chó này là quá hiếm và quá khác biệt so với những con chó khác đến mức chúng có khả năng phân loại chúng là bất thường (không có ý xúc phạm giống chó Chihuahua).



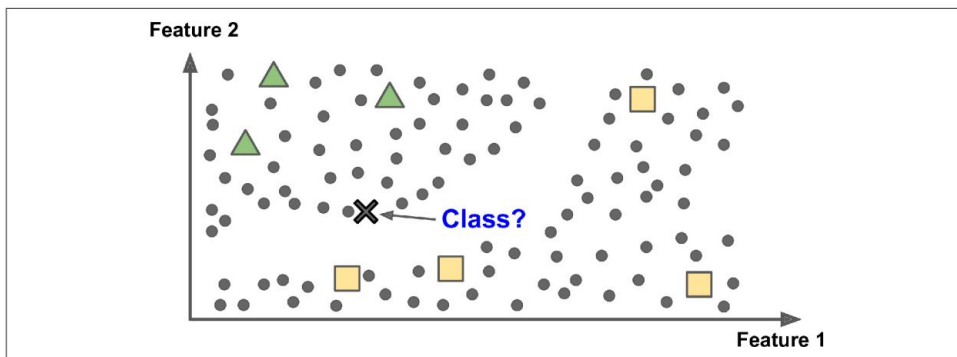
Hình 1-10. Phát hiện bất thường

Cuối cùng, một nhiệm vụ không giám sát phổ biến khác là học luật liên kết, trong đó mục tiêu là phân tích lượng lớn dữ liệu và khám phá các mối quan hệ thú vị giữa chúng.

thuộc tính. Ví dụ, giả sử bạn sở hữu một siêu thị. Việc chạy một quy tắc liên kết trên nhật ký bán hàng của bạn có thể tiết lộ rằng những người mua nước sốt thịt nướng và khoai tây chiên cũng có xu hướng mua bít tết. Do đó, bạn có thể muốn đặt những mặt hàng này gần nhau.

Học bán giám sát: Vì

việc gán nhãn dữ liệu thường tốn thời gian và chi phí, bạn thường sẽ có rất nhiều trường hợp chưa được gán nhãn và rất ít trường hợp đã được gán nhãn. Một số thuật toán có thể xử lý dữ liệu được gán nhãn một phần. Điều này được gọi là học bán giám sát (Hình 1-11).



Hình 1-11. Học bán giám sát với hai lớp (tam giác và hình vuông): các ví dụ chưa được dán nhãn (hình tròn) giúp phân loại một trường hợp mới (hình chữ thập) vào lớp tam giác thay vì lớp hình vuông, mặc dù nó gần với các hình vuông đã được dán nhãn hơn.

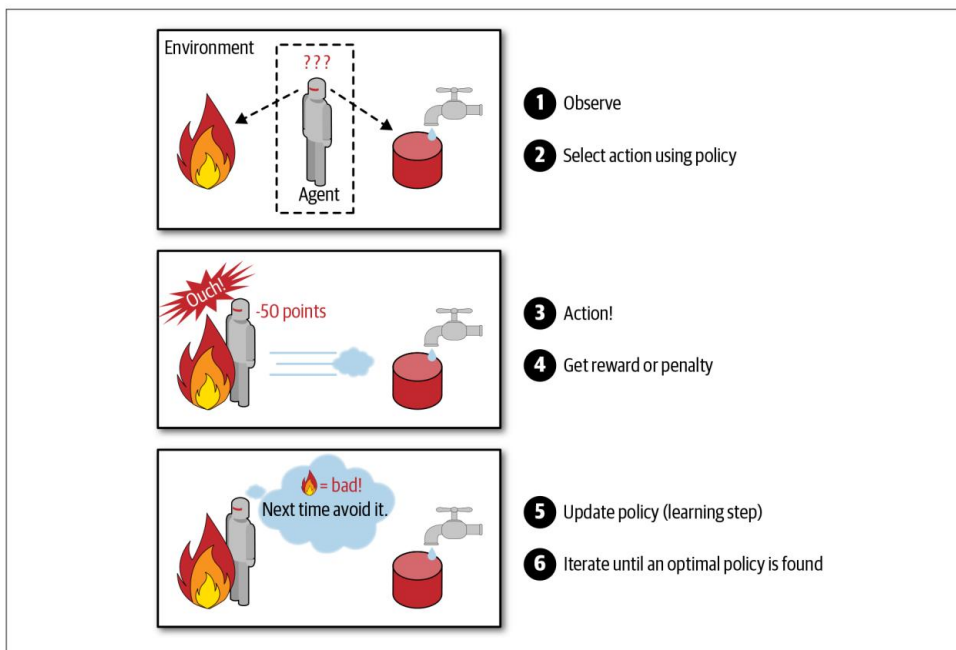
Một số dịch vụ lưu trữ ảnh, chẳng hạn như Google Photos, là những ví dụ điển hình. Sau khi bạn tải tất cả ảnh gia đình lên dịch vụ, hệ thống sẽ tự động nhận ra rằng cùng một người A xuất hiện trong ảnh 1, 5 và 11, trong khi một người khác B xuất hiện trong ảnh 2, 5 và 7. Đây là phần không giám sát của thuật toán (phân cụm). Bây giờ, tất cả những gì hệ thống cần là bạn cho nó biết những người này là ai. Chỉ cần thêm một nhãn cho mỗi người và hệ thống có thể đặt tên cho mọi người trong mọi bức ảnh, điều này rất hữu ích cho việc tìm kiếm ảnh.

Hầu hết các thuật toán học bán giám sát là sự kết hợp giữa các thuật toán học không giám sát và học có giám sát. Ví dụ, mạng niềm tin sâu (DBN) dựa trên các thành phần không giám sát được gọi là máy Boltzmann hạn chế (RBM) được xếp chồng lên nhau. Các RBM được huấn luyện tuần tự theo cách không giám sát, sau đó toàn bộ hệ thống được tinh chỉnh bằng các kỹ thuật học có giám sát.

4. Đó là khi hệ thống hoạt động hoàn hảo. Trên thực tế, nó thường tạo ra một vài nhóm cho mỗi người, và đôi khi nhầm lẫn hai người trông giống nhau, vì vậy bạn có thể cần cung cấp một vài nhãn cho mỗi người và tự tay dọn dẹp một số nhóm.

Học tăng cường

(Reinforcement Learning) là một lĩnh vực hoàn toàn khác. Hệ thống học tập, được gọi là tác nhân trong ngữ cảnh này, có thể quan sát môi trường, lựa chọn và thực hiện các hành động, và nhận được phần thưởng (hoặc hình phạt dưới dạng phần thưởng âm, như được minh họa trong Hình 1-12). Sau đó, nó phải tự học chiến lược tốt nhất, được gọi là chính sách, để nhận được phần thưởng cao nhất theo thời gian. Chính sách xác định hành động mà tác nhân nên chọn khi ở trong một tình huống nhất định.



Hình 1-12. Học tăng cường

Ví dụ, nhiều robot sử dụng thuật toán Học tăng cường (Reinforcement Learning) để học cách đi bộ. Chương trình AlphaGo của DeepMind cũng là một ví dụ điển hình về Học tăng cường: nó đã gây chú ý vào tháng 5 năm 2017 khi đánh bại nhà vô địch thế giới Ke Jie trong trò chơi cờ vây. Nó đã học được chiến lược chiến thắng bằng cách phân tích hàng triệu ván cờ, và sau đó tự chơi nhiều ván cờ với chính mình. Lưu ý rằng quá trình học đã bị tắt trong các ván đấu với nhà vô địch; AlphaGo chỉ đơn giản là áp dụng chiến lược mà nó đã học được.

Học theo lô và học trực tuyến Một

tiêu chí khác được sử dụng để phân loại các hệ thống Học máy là liệu hệ thống đó có thể học một cách tăng dần từ luồng dữ liệu đầu vào hay không.

Học theo lô:

Trong học theo lô, hệ thống không có khả năng học tăng dần: nó phải được huấn luyện bằng cách sử dụng tất cả dữ liệu có sẵn. Điều này thường tốn rất nhiều thời gian và tài nguyên tính toán, vì vậy nó thường được thực hiện ngoại tuyến. Đầu tiên, hệ thống được huấn luyện, sau đó được đưa vào sản xuất và chạy mà không học thêm nữa; nó chỉ áp dụng những gì đã học. Điều này được gọi là học ngoại tuyến.

Nếu bạn muốn một hệ thống học theo lô biết về dữ liệu mới (chẳng hạn như một loại thư rác mới), bạn cần huấn luyện một phiên bản mới của hệ thống từ đầu trên toàn bộ tập dữ liệu (không chỉ dữ liệu mới mà cả dữ liệu cũ), sau đó dừng hệ thống cũ và thay thế nó bằng hệ thống mới.

May mắn thay, toàn bộ quá trình huấn luyện, đánh giá và triển khai hệ thống Học máy có thể được tự động hóa khá dễ dàng (như thể hiện trong [Hình 1-3](#)), vì vậy ngay cả một hệ thống học theo lô cũng có thể thích ứng với sự thay đổi. Chỉ cần cập nhật dữ liệu và huấn luyện một phiên bản mới của hệ thống từ đầu thường xuyên khi cần thiết.

Giải pháp này đơn giản và thường hoạt động tốt, nhưng việc huấn luyện bằng toàn bộ tập dữ liệu có thể mất nhiều giờ, vì vậy thông thường bạn chỉ nên huấn luyện một hệ thống mới sau mỗi 24 giờ hoặc thậm chí chỉ mỗi tuần một lần. Nếu hệ thống của bạn cần thích ứng với dữ liệu thay đổi nhanh chóng (ví dụ: để dự đoán giá cổ phiếu), thì bạn cần một giải pháp phản ứng nhanh hơn.

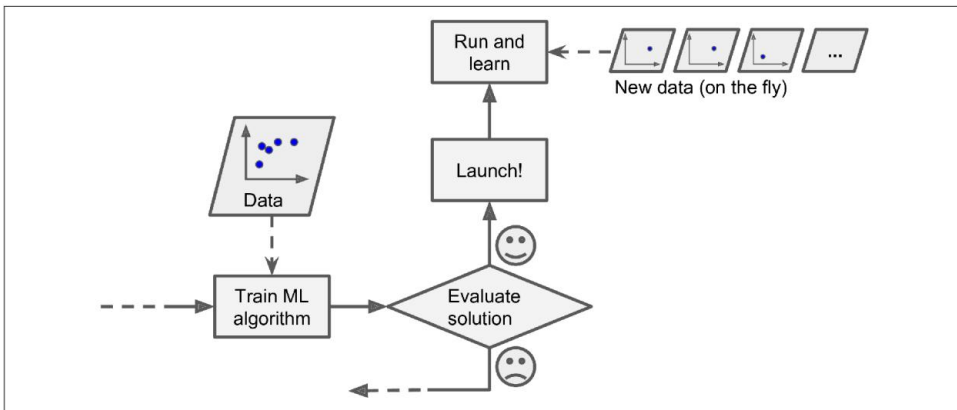
Ngoài ra, việc huấn luyện trên toàn bộ tập dữ liệu đòi hỏi rất nhiều tài nguyên tính toán (CPU, dung lượng bộ nhớ, dung lượng ổ đĩa, I/O ổ đĩa, I/O mạng, v.v.). Nếu bạn có nhiều dữ liệu và tự động hóa hệ thống để huấn luyện lại từ đầu mỗi ngày, cuối cùng sẽ tốn rất nhiều tiền. Nếu lượng dữ liệu quá lớn, thậm chí có thể không thể sử dụng thuật toán học theo lô.

Cuối cùng, nếu hệ thống của bạn cần có khả năng tự học và có nguồn lực hạn chế (ví dụ: ứng dụng trên điện thoại thông minh hoặc robot thám hiểm trên sao Hỏa), thì việc mang theo lượng lớn dữ liệu huấn luyện và tiêu tốn nhiều tài nguyên để huấn luyện hàng giờ mỗi ngày sẽ là một trở ngại lớn.

May mắn thay, trong tất cả các trường hợp này, lựa chọn tốt hơn là sử dụng các thuật toán có khả năng học hỏi một cách tăng dần.

Học trực tuyến

Trong học trực tuyến, bạn huấn luyện hệ thống một cách tăng dần bằng cách cung cấp cho nó các trường hợp dữ liệu theo trình tự, riêng lẻ hoặc theo nhóm nhỏ gọi là mini-batch. Mỗi bước học đều nhanh và tiết kiệm chi phí, do đó hệ thống có thể học về dữ liệu mới ngay lập tức khi dữ liệu được cung cấp (xem [Hình 1-13](#)).



Hình 1-13. Trong học trực tuyến, một mô hình được huấn luyện và đưa vào sử dụng thực tế, sau đó nó tiếp tục học hỏi khi có dữ liệu mới.

Học trực tuyến rất phù hợp với các hệ thống nhận dữ liệu liên tục (ví dụ: giá cổ phiếu) và cần thích ứng nhanh chóng hoặc tự động với sự thay đổi. Đây cũng là một lựa chọn tốt nếu bạn có tài nguyên máy tính hạn chế: một khi hệ thống học trực tuyến đã học được các trường hợp dữ liệu mới, nó không cần chúng nữa, vì vậy bạn có thể loại bỏ chúng (trừ khi bạn muốn có thể quay lại trạng thái trước đó và "phát lại" dữ liệu). Điều này có thể tiết kiệm một lượng lớn không gian lưu trữ.

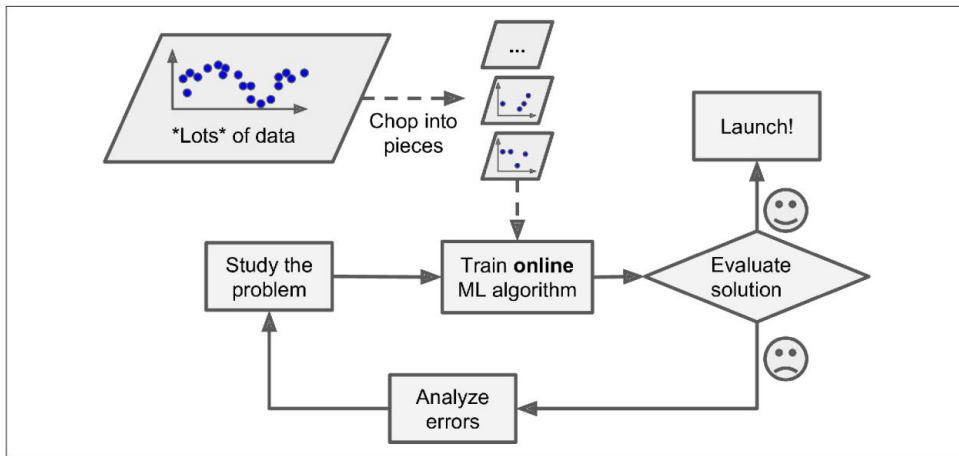
Các thuật toán học trực tuyến cũng có thể được sử dụng để huấn luyện hệ thống trên các tập dữ liệu khổng lồ mà không thể chứa vừa trong bộ nhớ chính của một máy (điều này được gọi là học ngoài bộ nhớ). Thuật toán tải một phần dữ liệu, chạy bước huấn luyện trên dữ liệu đó và lặp lại quá trình cho đến khi chạy trên toàn bộ dữ liệu (xem Hình 1-14).



Việc học ngoài chương trình giảng dạy thường được thực hiện ngoại tuyến (tức là không trên hệ thống trực tuyến), vì vậy thuật ngữ "học trực tuyến" có thể gây nhầm lẫn. Hãy coi đó như một hình thức học tập tăng dần.

Một thông số quan trọng của các hệ thống học trực tuyến là tốc độ thích ứng với dữ liệu thay đổi: điều này được gọi là tốc độ học tập. Nếu bạn đặt tốc độ học tập cao, hệ thống của bạn sẽ nhanh chóng thích ứng với dữ liệu mới, nhưng nó cũng sẽ nhanh chóng quên đi dữ liệu cũ (bạn không muốn bộ lọc thư rác chỉ gắn cờ những loại thư rác mới nhất mà nó nhận được).

Ngược lại, nếu bạn đặt tốc độ học thấp, hệ thống sẽ có quán tính lớn hơn; nghĩa là, nó sẽ học chậm hơn, nhưng cũng sẽ ít nhạy cảm hơn với nhiễu trong dữ liệu mới hoặc với các chuỗi điểm dữ liệu không đại diện (ngoại lệ).



Hình 1-14. Sử dụng học trực tuyến để xử lý các tập dữ liệu nhiễu loạn.

Một thách thức lớn đối với học trực tuyến là nếu dữ liệu xấu được đưa vào hệ thống, hiệu suất của hệ thống sẽ giảm dần. Nếu đó là một hệ thống đang hoạt động, khách hàng của bạn sẽ nhận thấy điều đó. Ví dụ, dữ liệu xấu có thể đến từ một cảm biến bị lỗi trên robot, hoặc từ ai đó spam công cụ tìm kiếm để cố gắng đạt thứ hạng cao trong kết quả tìm kiếm. Để giảm thiểu rủi ro này, bạn cần giám sát hệ thống của mình chặt chẽ và nhanh chóng tắt quá trình học tập (và có thể khôi phục lại trạng thái hoạt động trước đó) nếu phát hiện hiệu suất giảm sút. Bạn cũng có thể muốn giám sát dữ liệu đầu vào và phản ứng với dữ liệu bất thường (ví dụ: sử dụng thuật toán phát hiện bất thường).

Học dựa trên trường hợp cụ thể so với học dựa trên mô hình Một

cách khác để phân loại các hệ thống Học máy là dựa trên khả năng khái quát hóa của chúng.

Hầu hết các bài toán Học máy đều xoay quanh việc đưa ra dự đoán. Điều này có nghĩa là, với một số lượng ví dụ huấn luyện nhất định, hệ thống cần có khả năng đưa ra dự đoán tốt (khái quát hóa) cho các ví dụ mà nó chưa từng thấy trước đây. Có một chỉ số hiệu suất tốt trên dữ liệu huấn luyện là tốt, nhưng chưa đủ; mục tiêu thực sự là đạt hiệu suất tốt trên các trường hợp mới.

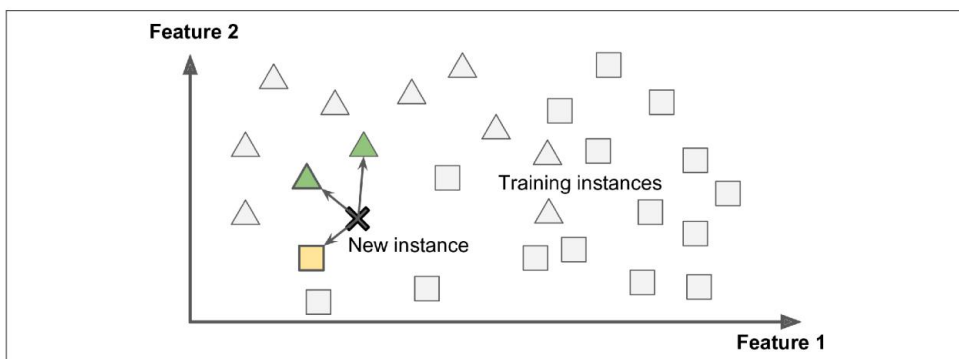
Có hai phương pháp chính để khái quát hóa: học dựa trên trường hợp cụ thể và học dựa trên mô hình.

Học tập dựa trên ví dụ Có

Lẽ hình thức học tập đơn giản nhất chính là học thuộc lòng. Nếu bạn tạo bộ lọc thư rác theo cách này, nó sẽ chỉ gắn cờ tất cả các email giống hệt với những email mà người dùng đã gắn cờ trước đó - không phải là giải pháp tối tệ nhất, nhưng chắc chắn không phải là tốt nhất.

Thay vì chỉ gắn cờ những email giống hệt với các email spam đã biết, bộ lọc spam của bạn có thể được lập trình để gắn cờ cả những email rất giống với các email spam đã biết. Điều này đòi hỏi một thước đo độ tương đồng giữa hai email. Một thước đo độ tương đồng (rất cơ bản) giữa hai email có thể là đếm số từ mà chúng có chung. Hệ thống sẽ gắn cờ một email là spam nếu nó có nhiều từ chung với một email spam đã biết.

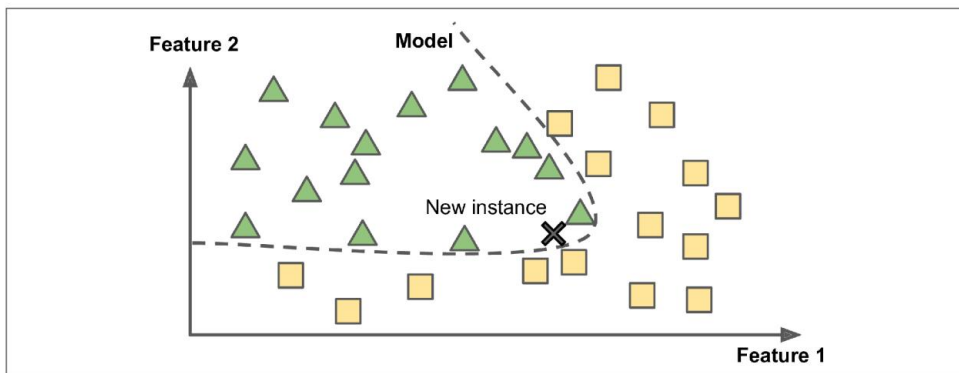
Đây được gọi là học dựa trên ví dụ: hệ thống học thuộc lòng các ví dụ, sau đó khái quát hóa cho các trường hợp mới bằng cách sử dụng thước đo độ tương đồng để so sánh chúng với các ví dụ đã học (hoặc một tập hợp con của chúng). Ví dụ, trong **Hình 1-15**, trường hợp mới sẽ được phân loại là hình tam giác vì phần lớn các trường hợp tương tự nhất thuộc về lớp đó.



Hình 1-15. Học tập dựa trên ví dụ

Học dựa trên mô hình

Một cách khác để khái quát hóa từ một tập hợp các ví dụ là xây dựng một mô hình từ các ví dụ này và sau đó sử dụng mô hình đó để đưa ra dự đoán. Điều này được gọi là học dựa trên mô hình (**Hình 1-16**).



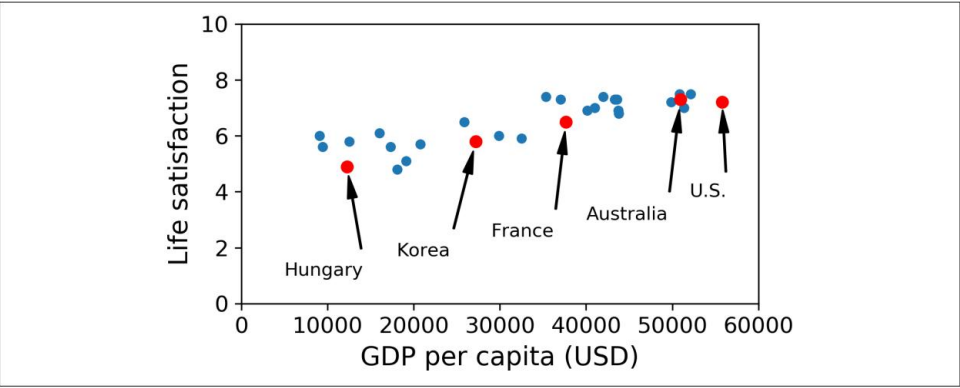
Hình 1-16. Học tập dựa trên mô hình

Ví dụ, giả sử bạn muốn biết liệu tiền có làm cho người ta hạnh phúc hay không, vậy nên bạn...
Tải dữ liệu Chỉ số Cuộc sống Tốt đẹp hơn từ [trang web của OECD](#). và số liệu thống kê về tổng số mái vòm-
Tổng sản phẩm quốc nội (GDP) bình quân đầu người được lấy từ [trang web của IMF](#). Sau đó bạn ghép các bảng lại với nhau và
Sắp xếp theo GDP bình quân đầu người. [Bảng 1-1](#) hiển thị một phần kết quả bạn nhận được.

Bảng 1-1. Tiền có làm cho con người hạnh phúc hơn không?

Quốc gia	GDP bình quân đầu người (USD)	Mức độ hài lòng về cuộc sống
Hungary	12.240	4.9
Hàn Quốc	27.195	5.8
Pháp	37.675	6,5
Úc	50.962	7.3
Hoa Kỳ	55.805	7.2

Hãy cùng vẽ biểu đồ dữ liệu cho các quốc gia này ([Hình 1-17](#)).



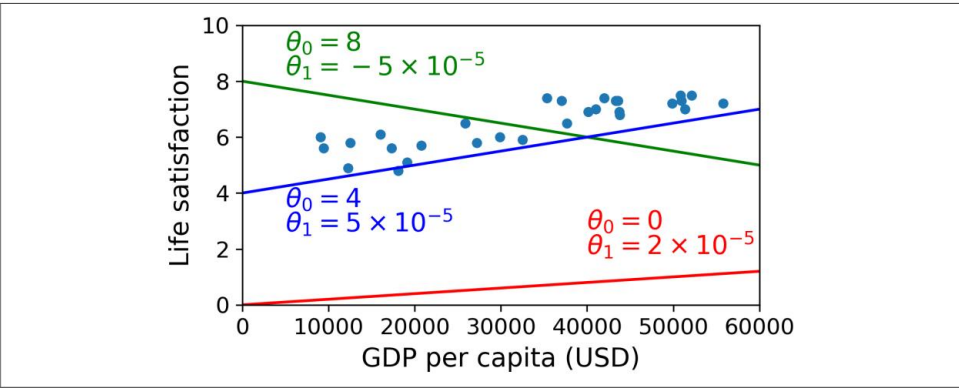
Hình 1-17. Bạn có nhận thấy xu hướng nào ở đây không?

Dường như có một xu hướng ở đây! Mặc dù dữ liệu có nhiều nhiễu (tức là một phần ngẫu nhiên), nhưng nó...
Có vẻ như mức độ hài lòng về cuộc sống tăng lên gần như tuyến tính khi GDP bình quân đầu người của quốc gia tăng.
Nó tăng lên. Vì vậy, bạn quyết định mô hình hóa sự hài lòng về cuộc sống như một hàm tuyến tính của GDP bình quân đầu người.
vốn. Bước này được gọi là lựa chọn mô hình: bạn đã chọn một mô hình tuyến tính về sự hài lòng trong cuộc sống.
chỉ với một thuộc tính duy nhất, GDP bình quân đầu người ([Phương trình 1-1](#)).

Phương trình 1-1. Một mô hình tuyến tính đơn giản

sự hài lòng về cuộc sống = $\theta_0 + \theta_1 \times \text{GDP bình quân đầu người}$

Mô hình này có hai tham số, θ_0 và θ_1 .⁵ Bằng cách điều chỉnh các tham số này, bạn có thể làm cho mô hình của mình biểu diễn bất kỳ hàm tuyến tính nào, như minh họa trong [Hình 1-18](#).



Hình 1-18. Một vài mô hình tuyến tính khả thi

Trước khi có thể sử dụng mô hình, bạn cần xác định các giá trị tham số θ_0 và θ_1 .
Làm thế nào bạn có thể biết giá trị nào sẽ giúp mô hình của bạn hoạt động tốt nhất? Để trả lời câu hỏi này, bạn cần xác định một thước đo hiệu suất. Bạn có thể định nghĩa một hàm tiện ích (hoặc hàm đánh giá) để đo lường mức độ tốt của mô hình, hoặc bạn có thể định nghĩa một hàm chi phí để đo lường mức độ kém hiệu quả của nó. Đối với các bài toán Hồi quy tuyến tính, người ta thường sử dụng một hàm chi phí để đo khoảng cách giữa các dự đoán của mô hình tuyến tính và các ví dụ huấn luyện; mục tiêu là giảm thiểu khoảng cách này.

Đây là lúc thuật toán Hồi quy tuyến tính phát huy tác dụng: bạn cung cấp cho nó các ví dụ huấn luyện, và nó sẽ tìm ra các tham số giúp mô hình tuyến tính phù hợp nhất với dữ liệu của bạn.
Quá trình này được gọi là huấn luyện mô hình. Trong trường hợp của chúng ta, thuật toán tìm ra các giá trị tham số tối ưu là $\theta_0 = 4,85$ và $\theta_1 = 4,91 \times 10^{-5}$.



Điều gây nhầm lẫn là, cùng một từ "mô hình" có thể dùng để chỉ một loại mô hình (ví dụ: Hồi quy tuyến tính), một kiến trúc mô hình được xác định đầy đủ (ví dụ: Hồi quy tuyến tính với một đầu vào và một đầu ra), hoặc mô hình đã được huấn luyện cuối cùng sẵn sàng để sử dụng cho dự đoán (ví dụ: Hồi quy tuyến tính với một đầu vào và một đầu ra, sử dụng $\theta_0 = 4,85$ và $\theta_1 = 4,91 \times 10^{-5}$). Lựa chọn mô hình bao gồm việc chọn loại mô hình và xác định đầy đủ kiến trúc của nó. Huấn luyện một mô hình có nghĩa là chạy một thuật toán để tìm các tham số mô hình sao cho nó phù hợp nhất với dữ liệu huấn luyện (và hy vọng sẽ đưa ra dự đoán tốt trên dữ liệu mới).

5. Theo quy ước, chữ cái Hy Lạp θ (theta) thường được sử dụng để biểu thị các tham số của mô hình.